

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

Направление (специальность)	27.03.04 - Управление в технических системах
Профиль (программа, специализация)	Системы и технические средства автоматизации и управления
Факультет/Институт	ФЭА
Кафедра	САУ

К защите допустить

Зав. кафедрой

Доброскок Н.А.

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
БАКАЛАВРА**

**Тема: РАЗРАБОТКА АЛГОРИТМА ДЛЯ АВТОМАТИЧЕСКОГО
ОБНАРУЖЕНИЯ ДЕФЕКТОВ ПРИ 3D-ПЕЧАТИ**

Студент	2491	_____	Белозор А.А.
		<i>Подпись</i>	
Руководитель	к. т. н., доцент (Уч. степень, уч. звание)	_____	Абдулаева З.М.
		<i>Подпись</i>	
Консультанты		_____	Гречухин М.Н.
	(Уч. степень, уч. звание)	<i>Подпись</i>	
		_____	Скрынская О.А.
	(Уч. степень, уч. звание)	<i>Подпись</i>	
		_____	Бельский Г.В.
	(Уч. степень, уч. звание)	<i>Подпись</i>	

Санкт-Петербург

2026

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

Утверждаю

Зав. кафедрой САУ

_____ Доброскок Н. А.

«__» _____ 2026 г.

Студент Белозор Алексей Александрович

Группа 2491

Тема работы: Разработка алгоритма для автоматического обнаружения дефектов при 3D-печати

Место выполнения ВКР: каф. САУ СПбГЭТУ «ЛЭТИ»

Исходные данные (технические требования):

Объект управления – FDM-принтер (Anycubic Kobra 2 Neo), видеопоток с веб-камеры Logitech C310. Классификация кадров на два класса: «норма» и «дефект экструзии». Требования: точность обнаружения дефекта $\geq 0,85$, ложные срабатывания $\leq 1\%$. Ограничения: загрузка CPU $\leq 60\%$, RAM ≤ 2 ГБ, частота ≥ 7 кадров/с, задержка ≤ 250 мс. Управление принтером через G-код (M25 – пауза, M24 – возобновление). Автономная система с графическим интерфейсом.

Содержание ВКР:

1. Анализ предметной области и обоснование выбора методов контроля качества.
2. Разработка и валидация модели машинного зрения.
3. Программно-аппаратная реализация системы автоматического управления.

Перечень отчетных материалов: ВКР (бумажная и электронная версии), компьютерная презентация

Дополнительные разделы: Экономическое обоснование ВКР

Дата выдачи задания
«__» _____ 2026 г.

Дата представления ВКР к защите
«__» _____ 2026 г.

Студент

Белозор А.А.

Руководитель к. т. н., доцент
(Уч. степень, уч. звание)

Абдулаева З.М.

Консультант

(Уч. степень, уч. звание)

Гречухин М.Н.

КАЛЕНДАРНЫЙ ПЛАН ВЫПОЛНЕНИЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

Утверждаю

Зав. кафедрой САУ

_____ Доброскок Н. А.

«__» _____ 2026 г.

Студент Белозор Алексей Александрович

Группа 2491

Тема работы: Разработка алгоритма для автоматического обнаружения дефектов при 3D-печати

№ п/п	Наименование работ	Срок выполнения
1	Обзор литературы по теме работы	16.03 – 22.03
2	Анализ предметной области и выбор методов контроля качества	23.03 – 05.04
3	Разработка и валидация модели машинного зрения	06.04 – 26.04
4	Программно-аппаратная реализация системы управления	27.04 – 17.05
5	Экономическое обоснование	18.05 – 24.05
6	Оформление пояснительной записки	25.05 – 28.05
7	Оформление иллюстративного материала	28.05 – 30.05

Студент _____ Белозор А.А.

Руководитель _____ К. Т. Н., доцент
(Уч. степень, уч. звание) Абдулаева З.М.

Консультант _____ Гречухин М.Н.
(Уч. степень, уч. звание)

РЕФЕРАТ

Пояснительная записка содержит 62 с., 10 рис., 18 табл., 18 ист.

Ключевые слова: АДДИТИВНОЕ ПРОИЗВОДСТВО, FDM-ПЕЧАТЬ, КОНТРОЛЬ КАЧЕСТВА, ОБНАРУЖЕНИЕ ДЕФЕКТОВ ЭКСТРУЗИИ, КОМПЬЮТЕРНОЕ ЗРЕНИЕ, НЕЙРОННАЯ СЕТЬ YOLOv8, БИНАРНАЯ КЛАССИФИКАЦИЯ, АВТОМАТИЧЕСКОЕ УПРАВЛЕНИЕ, ПОСЛЕДОВАТЕЛЬНЫЙ ИНТЕРФЕЙС USB/UART, МНОГОПОТОЧНАЯ АРХИТЕКТУРА, ТРАНСФЕРНОЕ ОБУЧЕНИЕ, РЕАЛЬНОЕ ВРЕМЯ.

Объект исследования – процесс FDM-печати.

Предмет исследования – программно-алгоритмическое обеспечение визуального контроля и управления 3D-принтером на базе сверточной нейросети.

Цель работы – разработка автономной системы контроля качества 3D-печати, детектирующей дефекты экструзии в реальном времени и управляющей оборудованием.

Методы исследования – трансферное обучение, аугментация данных, оптимизация гиперпараметров, многопоточная архитектура (PyQt6, QThread), протокол G-код, технико-экономическое обоснование.

Проведен анализ существующих решений, обоснован выбор архитектуры нейросети. Сформирован датасет из 46 664 изображений, выполнены предобработка и аугментация, проведено обучение модели. Разработан графический интерфейс и модуль управления принтером, выполнена интеграция компонентов. Достигнута точность классификации 99,71% и точность обнаружения дефектов 99,82%, испытания подтвердили задержку реакции не более 250 мс и нагрузку процессора 40–55%.

Результаты – готовый к эксплуатации программный комплекс, способный обнаруживать дефекты экструзии, автоматически останавливать печать и возобновлять процесс после устранения проблемы.

ABSTRACT

Under the influence of thermal fluctuations and filament inconsistencies, FDM 3D printing processes are highly susceptible to extrusion defects, which lead to material waste, equipment wear, and prolonged idle time. Moreover, existing monitoring solutions based on cloud AI services or open-source plugins often suffer from network latency, closed architectures, or excessive hardware requirements, limiting their applicability in isolated or resource-constrained environments. Therefore, a local, autonomous computer vision system leveraging a lightweight convolutional neural network and a multi-threaded control architecture will be considered to identify a more efficient and reliable approach for real-time defect detection and printer management.

This paper presents the development and validation of a closed-loop monitoring system based on the YOLOv8n-cls architecture, integrated with a PyQt6 graphical interface and USB/UART communication for safe execution of G-code commands (M25/M24). The proposed solution demonstrates 99.71% classification accuracy, maintains a processing frequency of 7.5–8.0 FPS on standard hardware, and significantly reduces material losses through automated pause/resume functionality.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ВЫБОР МЕТОДОВ КОНТРОЛЯ КАЧЕСТВА	13
1.1 Обзор существующих решений и анализ аналогов	13
1.2 Требования к разрабатываемой системе контроля качества	17
1.3 Сравнительный анализ архитектур нейронных сетей для классификации изображений	21
1.3.1 Постановка задачи классификации	21
1.3.2 Обзор кандидатов и экспериментальное сравнение	23
1.3.3 Обоснование выбора YOLOv8n-cls	23
1.4 Теория управления по последовательному интерфейсу и командному языку G-код	25
2 РАЗРАБОТКА И ВАЛИДАЦИЯ МОДЕЛИ МАШИННОГО ЗРЕНИЯ	29
2.1 Формирование, предобработка и аугментация датасета	29
2.2 Методика обучения и оптимизация гиперпараметров	32
2.3 Валидация модели и оценка производительности	36
3. ПРОГРАММНО-АППАРАТНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ АВТОМАТИЧЕСКОГО УПРАВЛЕНИЯ	39
3.1 Архитектура системы и организация контура управления	39
3.2 Программная реализация последовательного интерфейса USB/UART	43
3.3 Интеграция программных модулей и проверка устойчивости системы	45
4 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ	49
4.1 Расчет затрат на оплату труда	49
4.2 Отчисления на социальные нужды	52

4.3 Материалы	53
4.4 Амортизационные отчисления	53
4.5 Услуги сторонних организаций.....	55
4.6 Накладные расходы	55
4.7 Смета затрат.....	55
ЗАКЛЮЧЕНИЕ	58
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	61

ВВЕДЕНИЕ

Актуальность темы. Аддитивные технологии, в частности FDM-печать, активно внедряются как в современные производства, так и в домохозяйства. В целом отмечается значительный рост всей отрасли 3D-печати, охватывающей услуги, материалы, оборудование и программное обеспечение. Однако для дальнейшего устойчивого развития отрасли возникает необходимость решения ряда технологических задач.

Одной из таких задач является прекращение печати при выявлении дефекта на изготавливаемом изделии. Даже качественная калибровка оборудования не может гарантировать воспроизведения объекта без изъянов, так как качество печати остается чувствительным к вариациям температуры и качества филамента. Поэтому своевременное выявление брака позволяет избежать перерасхода материала на уже деформированную модель и потери времени, затрачиваемого на завершение заведомо некачественного изделия.

Ручной контроль качества требует непрерывного наблюдения за процессом со стороны оператора, невозможного при длительных печатях, что позволяет сделать вывод о низкой эффективности метода в современных условиях.

Альтернативно последние 5-7 лет развиваются решения на базе искусственного интеллекта и компьютерного зрения. Пионерами в этой области стали облачные сервисы, такие как The Spaghetti Detective (в настоящий момент Obico), PrintWatch и Sentry. Данные системы используют камеры для мониторинга и алгоритмы машинного обучения для выявления отклонений печати. Массовость их интеграции в локальные производства обусловлена рядом существенных ограничений, таких как:

- **Зависимость от интернет соединения и доступности облачной инфраструктуры.** Облачная инфраструктура требует доступа в интернет (что является сильным ограничительным фактором, выражающемся в физической

невозможностью подключения, в отсутствии стабильного соединения, изолированностью сети), передача видеопотока на удаленные серверы создает задержки. Так же возникает риск утечки данных.

- **Отсутствие прямого аппаратного управления.** Большинство существующих решений выполняют лишь функцию уведомления пользователя в мессенджере или почте, что не решает проблему полностью так как требует вмешательства человека.

- **Высокая стоимость и закрытая архитектура.** Подписки на облачные сервисы или покупка специализированных аппаратных модулей зачастую нецелесообразны для малых предприятий, образовательных учреждений и частных пользователей. А отсутствие исходного кода блокирует возможность адаптации под специфическую аппаратную часть или дефект, свойственный конкретной модели.

- **Сложность интеграции с локальным ПО.** Open-source альтернативы, такие как плагин Moonraker-Timelapse, скрипты для OctoPrint, тоже имеют ряд недостатков. Они требуют сложной настройки Linux-среды (Raspberry Pi), не обеспечивают необходимой производительности на слабых устройствах или не имеют удобного графического интерфейса для настройки порогов чувствительности в реальном времени.

На основании вышеизложенного можно определить объективную потребность в реализации локальной, автономной системы визуального контроля 3D-печати, способной в реальном времени детектировать аномалии и управлять производственным оборудованием без участия человека.

Целью работы является разработка автономной и прозрачной системы контроля качества 3D-печати на основе компьютерного зрения, позволяющей автоматического обнаружить дефекты в реальном времени и управлять принтером для предотвращения брака, перерасхода материальных и временных ресурсов.

Объект исследования – процесс аддитивного производства методом FDM-печати.

Предмет исследования – программно-алгоритмическое обеспечение автоматического визуального контроля и управления 3D-принтером на базе сверточной нейронной сети.

Для достижения поставленной цели были решены следующие задачи:

1. Проведен анализ существующих методов контроля качества в 3D-печати и обоснован выбор легковесной архитектуры нейросети для решения проблемы бинарной классификации.

2. Был сформирован датасет, проведена его очистка от артефактов, выполнена аугментация данных и достигнут баланс между классами «норма» и «дефект экструзии».

3. Обучена модель с подобранными гиперпараметры и честно произведена оценка качества распознавания на подготовленной выборке.

4. Написан модуль обмена данными по USB/UART, изучены правила протокола G-код и запрограммирован алгоритм безопасной остановки.

5. Спроектирован многопоточный графический интерфейс на PyQt6: обеспечено отображение видеопотока, настройка порогов срабатывания и ведение журнала событий.

6. Совершено объединение модулей, произведено полноценное тестирование и рассчитана экономическая выгода проекта.

Удалось успешно адаптировать компактную YOLOv8n-cls под задачу детекции дефектов экструзии при условии ограниченности вычислительных ресурсов. Введена предобработки входящих в модель данных: центральный квадратный кроп, перевод в градации серого с последующим дублированием канала в RGB-формат. Последнее необходимо из-за особенностей предобученной модели. В совокупности это позволило сформировать устойчивость работы, без вмешательства в базовую архитектуру. Строение локального за-

мкнутого контура полностью исключает облачные задержки. Обмен с микроконтроллером принтера реализован на асинхронном handshake-протоколе, что делает систему отказоустойчивой.

Практическая ценность проекта заключается полноценной реализации готового программного комплекса, который объективно способен снижать расход материалов, амортизацию оборудования и временные издержки. Все это позволяет избавиться от архаичного ручного метода при на фоне высокого уровня прозрачности, автономности и адаптивности проекта, под самые разные типы задач и оборудования. Интерфейс является интуитивно понятным, что позволяет пользователю быстро освоиться без необходимости изучения многоуровневой системы вкладок и параметров. Отсутствует необходимость специальных навыков программирования для запуска и использования реализованного решения. Система является легко интегрируемой в различные автоматизированные системы управления. Расчеты подтверждают экономическую эффективность: на одном принтере затраты окупаются за 3-4 месяца, в случае масштабирования решения на парк из 3-5 устройств, срок сокращается почти до одного месяца.

В работе был произведен теоретический анализ литературы и протоколов ЧПУ. В машинном обучении применялись следующие методы: transfer learning, аугментация данных, кросс-валидация. При проектировании программного обеспечения были учтены особенности событийно-ориентированной архитектуры, многопоточности и работы с последовательными портами. Экспериментальная часть включала оценку метрик классификации, замеры задержек инференса, стресс-тесты на отказоустойчивость. Экономическая обоснованность проекта выполнена согласно общепринятым стандартам для инженерных разработок.

Структура ВКР: введение, 4 главы, заключение, список источников и приложения. Во введении обоснована актуальность, сформулированы цель, задачи, объект и предмет исследования. Первая глава - анализ предметной области, выбор YOLOv8n-cls, формирование датасета и оценка метрик обучения.

Вторая глава - разбор тонкостей взаимодействия с принтером (последовательный интерфейс, G-код, прошивка микроконтроллеров), описан модуль безопасного управления. Третья - проектирование многопоточного графического интерфейса на PyQt6, принципы взаимодействия компонентов и отказоустойчивости. В четвертой главе приведено экономическое обоснование: себестоимость, затраты на материалы, услуги, амортизацию, целесообразность разработки. В выводе подведены итоги с перспективами развития и улучшения системы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ВЫБОР МЕТОДОВ КОНТРОЛЯ КАЧЕСТВА

1.1 Обзор существующих решений и анализ аналогов

Аддитивное производство давно вышло за пределы любительских мастерских. Вместе с ростом объемов печати выросла и потребность в автоматизации контроля качества. Сегодня рынок предлагает несколько готовых платформ, способных отслеживать процесс послойного наплавления и реагировать на отклонения. На первый взгляд выбор кажется очевидным: есть проверенные open-source решения, облачные сервисы с готовыми моделями и аппаратные AI-модули. Но при ближайшем рассмотрении становится ясно, что универсального инструмента не существует. Каждое решение закрывает только часть задач, оставляя за собой системные ограничения.

Один из самых распространенных вариантов – экосистема на базе **OctoPrint**. По своей сути это веб-сервер, разворачиваемый на одноплатном компьютере (чаще всего Raspberry Pi). Платформа берет на себя управление принтером, ведение логов, запуск таймлапсов и работу с очередью заданий. Для детекции дефектов обычно подключают сторонние плагины или самостоятельные скрипты, взаимодействующие с камерой. Схема архитектуры OctoPrint с плагинами приведена на рисунке 1.1. Примечательно, что OctoPrint не содержит встроенного ИИ. Это каркас, к которому исследователи и энтузиасты прикручивают свои алгоритмы. Плюсы очевидны: открытая архитектура, низкая стоимость базового железа, активное сообщество и десятки интеграций. Минусы тоже серьезные. Вычислительные ресурсы одноплатников сильно ограничены. Запуск нейросети в реальном времени без отставания кадра – задача нетривиальная. Часто инференс выносят на отдельный сервер или в облако, что сразу добавляет задержку и ставит под вопрос безопасность

принтер» добавляет от 500 мс до нескольких секунд. Для медленных процессов это иногда допустимо, но в динамичных сценариях печать уже успеет уйти в брак. В-третьих, подписка. Бесплатный тариф сильно ограничен, а коммерческий вариант превращает систему в регулярные операционные расходы. Стоит отметить и то, что пользователь не владеет ни моделью, ни сырыми данными. Все остается на серверах компании.

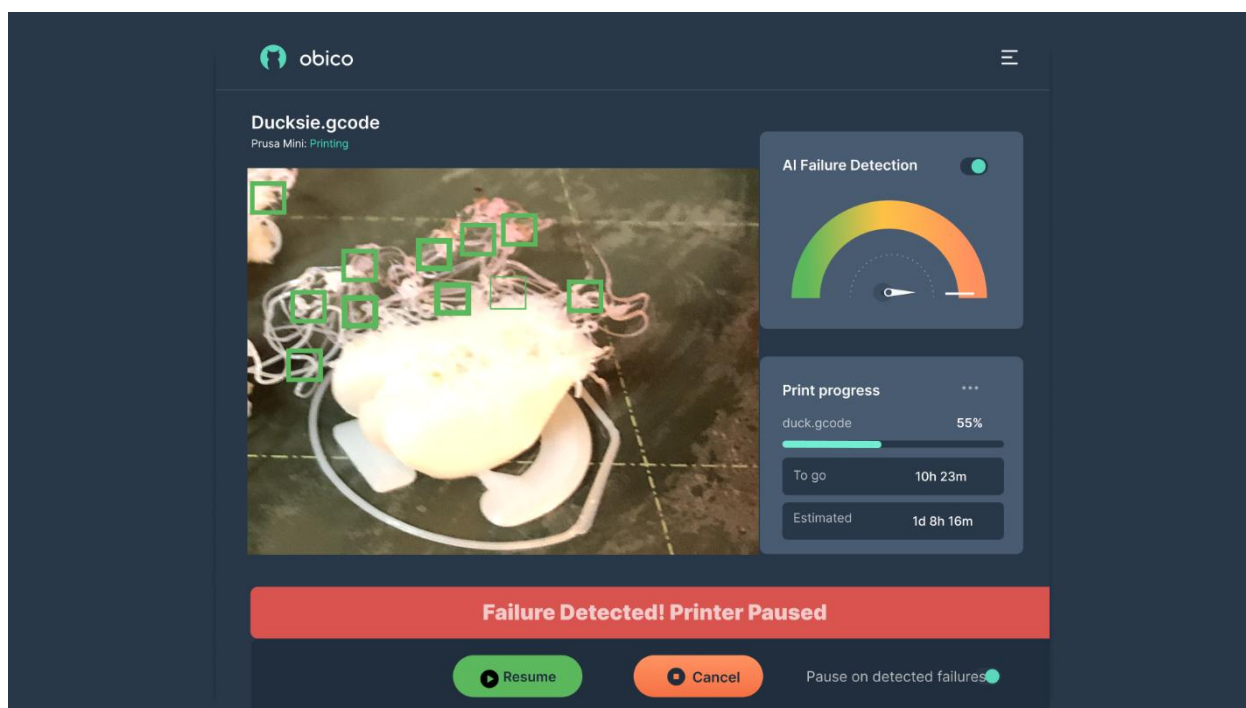


Рисунок 1.2 – Скриншот интерфейса Obico с детекцией дефекта

Третья группа решений – **коммерческие AI-камеры** и аппаратные модули (например, Crealiti Smart AI Camera, независимые Edge-устройства от профильных стартапов). Это уже не софт, а готовое устройство со встроенным процессором и оптимизированной нейросетью. Камера крепится рядом с экструдером, обрабатывает кадр локально и через UART или Wi-Fi отправляет команду остановки. Плюсы: минимальная задержка, полная автономность, отсутствие облачных рисков. Минусы: закрытая проприетарная экосистема. Пользователь не может изменить архитектуру модели, настроить пороги под свои материалы или экспортировать логи в свою систему аналитики. Стоимость тоже заметна – от 8 до 20 тысяч рублей за модуль, плюс часто требуется

привязка к конкретным брендам принтеров. Нередко такие камеры работают только с определенными версиями прошивок или требуют кастомных патчей.

Чтобы структурировать сравнение, удобно свести ключевые параметры в таблицу.

Таблица 1.1 – Сравнительный анализ существующих решений

Решение	Архитектура	Стоимость	Открытость / Гибкость	Зависимость от сети
OctoPrint + плагины	Локальный сервер (RPi / ПК)	Низкая (железо ~5–15 тыс. руб.)	Высокая (open-source, полная кастомизация)	Умеренная (требуется для удаленного доступа)
Obico (Spaghetti Detective)	Облачный инференс	Подписка (от 0 до ~10 \$/мес)	Низкая (закрытая модель, ограниченный API)	Высокая (обязательный стабильный интернет)
Коммерческие AI-камеры	Встроенный Edge-модуль	Высокая (8–20 тыс. руб. за модуль)	Нулевая / низкая (проприетарный софт, закрытая прошивка)	Отсутствует или минимальна

Анализ показывает интересную закономерность. Чем выше автономность и ниже задержка, тем больше система упирается в стоимость и закрытость. Чем доступнее и более открытым является решение, тем сильнее оно зависит от внешних сетей или требует значительных вычислительных ресурсов на стороне пользователя. На практике это создает разрыв между потребностью лабораторий и мелкосерийных участков в надежном локальном контроле и реальными возможностями массовых продуктов.

Именно этот разрыв определяет задачу дипломного проекта. Нужно предложить архитектуру, которая сочетает локальную обработку без облачных задержек, открытую настраиваемую модель, умеренные требования к железу и возможность интеграции с существующим оборудованием без покупки специализированных камер. Решение должно работать на обычном ПК, сохранять точность, достаточную для реального производства, и не требовать постоянного подключения к интернету.

1.2 Требования к разрабатываемой системе контроля качества

Перед началом реализации данного проекта были выделены следующие требования. Программа должна анализировать видеопоток в реальном времени. Изначально модель должна была реагировать даже на слабовыраженные дефекты, однако в процессе формирования датасета, этот пункт был пересмотрен и заменен на более мягкий – модель должна реагировать на ярко выраженные аномалии печати.

Для более гибкой настройки предусмотрено изменение порога уверенности, который регулируется ползунком интерфейса, диапазон настройки охватывает значения от пятидесяти до ста процентов, значение по умолчанию – восемьдесят. Основные требования к точности классификации и устойчивости системы формализованы и представлены в таблице 1.2.

Производительность программы в большей мере определяется ресурсами рабочей вычислительной машины. В нейросеть поступает тензор размеров триста двадцать на триста двадцать пикселей, полученное после предобработки кадра полученного с веб-камеры. Нагрузка на центральный процессор стабилизируется в диапазоне 40–55% при работе на процессорах уровня AMD Ryzen 5 5600U. Этот показатель можно регулировать, изменяя в коде частоту подачи изображений в модель. Потребление оперативной памяти стабилизируется в пределах трехсот мегабайт. Так же алгоритм намеренно анализирует не каждый кадр, а каждый второй, что позволяет снизить нагрузку. Детальные

ограничения по производительности и способы их программной реализации сведены в таблицу 1.3.

Таблица 1.2 – Формализованные требования к точности и устойчивости

№	Требование	Тип	Метод проверки	Критерий выполнения
ТР-1	Точность классификации (Ассигасу)	Функциональное	Тест на отложенную выборку	$\geq 90\%$
ТР-2	Точность в классе «дефект»	Функциональное	Расчет по матрице ошибок	$\geq 0,85$
ТР-3	Устойчивость к изменению освещения	Надежность	Тест с изменением яркости на $\pm 5\%$	Падение показателей $\leq 5\%$
ТР-4	Отсутствие ложных срабатываний	Надежность	Тест на видеопечатной печати	Ложные срабатывания $\leq 1\%$ кадров

Таблица 1.3 – Требования к производительности и ресурсопотреблению

Параметр	Ограничение	Способ обеспечения в ПО
Загрузка CPU	$\leq 60\%$	Пропуск четных кадров, <code>imgsz=320</code> <code>time.sleep(0.01)</code>
Потребление RAM	≤ 2 ГБ	Асинхронное QThread, копирование массивов только перед отправкой сигнала
Частота inference	7–8 КАДРОВ В СЕКУНДУ	<code>frame_idx % 2 != 0</code> , отключение <code>verbose</code> в Ultralytics
Задержка отрисовки GUI	< 50 мс	QImage без промежуточных конвертаций, SmoothTransformation

Такие значения являются осознанными результатами инженерных компромиссов. Выбор тензора 320x320 является быстрым вариантом для nano-моделей(YOLOv8n), тензор в 640x640 увеличил бы точность детекции, но вместе с этим значительно выросла бы и нагрузка на процессор. Это сделало бы невозможным использование программы на более слабом железе, что введет излишние ограничения и не позволит использовать программу на широком спектре оборудования, включая маломощные системы. Уменьшение до 160, наоборот, снизило бы нагрузку, в ущерб точности детекции, что тоже является нежелательным. Сравнительный анализ приведен в таблице 1.4.

Таблица 1.4 – Сравнительный анализ производительности и точности модели YOLOv8n-cls при изменении размера входного тензора

imgsz	Время инференса (CPU), мс	Детализация дефектов	Загрузка CPU, %	Оценка
224	25–35	Умеренная. Мелкие дефекты могут утрачиваться.	35–45	Допустимо при ограниченных вычислительных ресурсах.
320	40–50	Высокая. Достаточно для детекции расслоений и нарушений экструзии.	40–55	Наиболее целесообразный вариант. Баланс точности и скорости.
480	80–100	Высокая. Прирост точности незначителен.	65–85	Избыточно для задачи бинарной классификации.
640	15–200	Максимальная.	90 –100	Нецелесообразно. Критическое снижение частоты анализа.

Нагрузка на процессор определена следующими операциями: захват и декодирование видео, предобработка кадра, инференс YOLOv8n-cls, отрисовка графического интерфейса для пользователя, опрос COM-порта, интерпретатор Python. Поэтому такая нагрузка носит вполне оправданный характер, а оставшиеся мощности гарантируют стабильную работу операционной системы и параллельное выполнение нетребовательных к ресурсам приложений.

Интерфейс соответствует постулатам эргономики. Темная палитра снижает зрительное утомление оператора. Левая панель содержит элементы управления в области, соответствующей зоне первичного внимания. Правая область отведена видеопотоку и занимает основную площадь окна. Переключение режимов реакции выполняется путем выбора элемента из выпадающего списка. Цвет текста, отражающего состояние подключения принтера, меняется динамически. Красный цвет сообщает об обрыве связи, зеленый – подтверждает успешное соединение с оборудованием. Выбранная цветовая схема опирается на общепринятые семантические паттерны, что делает индикацию интуитивно понятной.

Диагностируемость была достигнута за счет централизованного логирования, каждое событие имеет временную ветку, совместно эти данные сохраняются в папке logs. Параллельно они передаются в консоль событий, которая автоматически прокручивается к последней записи. Предусмотрены различные маркеры и подписи информирующие о различных сбоях, например, ошибки загрузки модели или отказы отправки команд. Архив журнала позволяет провести анализ причин уже совершенной остановки. Функциональные требования к системе логирования и соответствующие методы их программной реализации обобщены в таблице 1.5.

Безопасность управления осуществляется жесткой фильтрацией сигналов. Программа сверяет показатель текущей уверенности с заданным оператором порогом. Интервал между повторными реакциями ограничен двумя секундами. Микроконтроллер получает команду M25 и принтер мгновенно ставит SD-печать на программную паузу. Так же происходит асинхронный опрос

порта каждые сто миллисекунд, исключая блокировку главного цикла. Корректное завершение работы гарантирует предотвращение блокировки камеры при повторном подключении за счет освобождения ее системных дескрипторов.

Таблица 1.5 – Требования к логированию и диагностируемости

Функция	Реализация	Цель
Временные метки	<code>time.strftime("%H:%M:%S")</code>	Привязка событий к хронологии печати
Автопрокрутка консоли	<code>cursor.movePosition(End)</code>	Просмотр актуального состояния без скроллинга
Сохранение в файл	Открытый поток <code>open("log.txt", "a", encoding="utf-8")</code>	Постоянное хранение истории запусков
Маркировка ошибок	try-except блоки в <code>CameraThread</code>	Быстрая локализация сбоев кода
Изоляция потоков	<code>pyqtSignal + frame_ready.emit()</code>	Предотвращение гонок данных и сбоев графического интерфейса

Технические требования задают рабочие рамки проекта. Нейросеть реагирует на выявленные аномалии, механизм безопасной остановки управляет физическим процессом печати. Надежность гарантируется асинхронной архитектурой программы, двойное логирование обеспечивает полную прозрачность действий. Наличие графического интерфейса снижает порог входа для использования предлагаемого решения. Перечисленные пункты легли в основу проектных решений следующих глав.

1.3 Сравнительный анализ архитектур нейронных сетей для классификации изображений

1.3.1 Постановка задачи классификации

Система должна определять только факт наличия дефекта на текущем кадре. Решение задачи локализации аномалии не является приоритетом, по-

скольку потребовало бы существенных вычислительных затрат. Следовательно, поставленная задача относится к классу задач бинарной классификации изображений. Можно выделить два принципиальных пути для создания классификатора. Первый – проектирование сверточной сети с нуля, инженеру требуется самостоятельно определить структуру, задать количество слоев, типы активаций и топологию связей. Такой метод требует очень обширной базы, размеченных примеров и недель вычислений на GPU, и глубоких компетенций в области нейросетевых архитектур. Второй подход – трансферное обучение, выбор уже сформированной модели, предварительно натренированной на миллионах изображений общего назначения. Сравнительная характеристика основных архитектурных кандидатов представлена в таблице 1.6.

Таблица 1.6 – Сравнительная характеристика архитектур сверточных сетей для задачи бинарной классификации изображений

Модель	Параметры (млн)	FLOPS (G)	Точность (%)	Адаптация под задачу	Примечание
ResNet-50	25.6	4.1	76.1	Требует замены классификационной головы	Избыточна по вычислительной сложности
EfficientNet-B0	5.3	0.39	77.1	Хорошая масштабируемость	Чувствительна к шумам входного кадра
MobileNetV3-Small	2.9	0.06	68.5	Компактная, но низкая точность	Недостаточна для детекции текстурных дефектов
YOLOv8n-cls	2.7	0.8	~74.5	Готовая классификационная голова, поддержка transfer learning	Оптимальный баланс точности и скорости

В этом случае ранние слои уже научились выделять ребра, градиенты и текстуру и задача сводится к адаптации последних блоков под конкретную выборку. Такой путь экономит ресурсы и значительно снижает порог входа в разработку.

1.3.2 Обзор кандидатов и экспериментальное сравнение

Выбор конкретной предобученной сети диктуется ограничениями целевой платформы. Тяжелые архитектуры вроде ResNet-50 показывают высокую точность на эталонных наборах данных, однако взамен растет количество параметров, что приводит к увеличению вычислительной нагрузки. Также появляется проблема переполнения оперативной памяти из-за скапливания промежуточных тензоров. Компактные варианты вроде MobileNetV3-Small обрабатывают данные существенно быстрее, потребление ресурсов остается минимальным, но это достигается за счет потери детальности. Экспериментальные замеры потребления вычислительных ресурсов сведены в таблицу 1.7.

Таблица 1.7 – Результаты экспериментального сравнения вычислительной нагрузки на целевой платформе

Модель	Время обработки (CPU), мс	Потребление VRAM, МБ	Итоговая оценка применимости
ResNet-50	320–450	~450	Нецелесообразно для развертывания на CPU: избыточная вычислительная сложность при незначительном приросте точности для бинарной задачи.
EfficientNet-B0	180–280	~220	Ограниченно применима: требует GPU для стабильной работы в реальном времени, чувствительна к шумам входного кадра.
MobileNetV3-Small	60–90	~70	Нецелесообразно: недостаточная глубина архитектуры
YOLOv8n-cls	40–50	~90	Оптимальный вариант: обеспечивает баланс между точностью классификации, потреблением ресурсов и скоростью обработки.

Две вышеприведенные крайности помогают прийти к заключению, что оптимальная модель должна быть достаточно легкой, для обеспечения работы в реальном времени, и при этом достаточно чувствительной, чтобы замечать текстурные аномалии экструзии.

1.3.3 Обоснование выбора YOLOv8n-cls

Анализируя предыдущее можно с уверенностью сказать, что архитектура YOLOv8n-cls полностью удовлетворяет требованиям. На рисунке 1.3 отображена последовательность этапов трансферного обучения. Веса основной части заблокированы для изменения на время дообучения, слои отвечающие за классификацию изменяются в соответствии с новым специфичным датасетом с двумя классами, в итоге получаем адаптированную под нашу задачу нейронную сеть. Подобная стратегия радикально экономит вычислительные и временные ресурсы на этапе определения весовых коэффициентов.

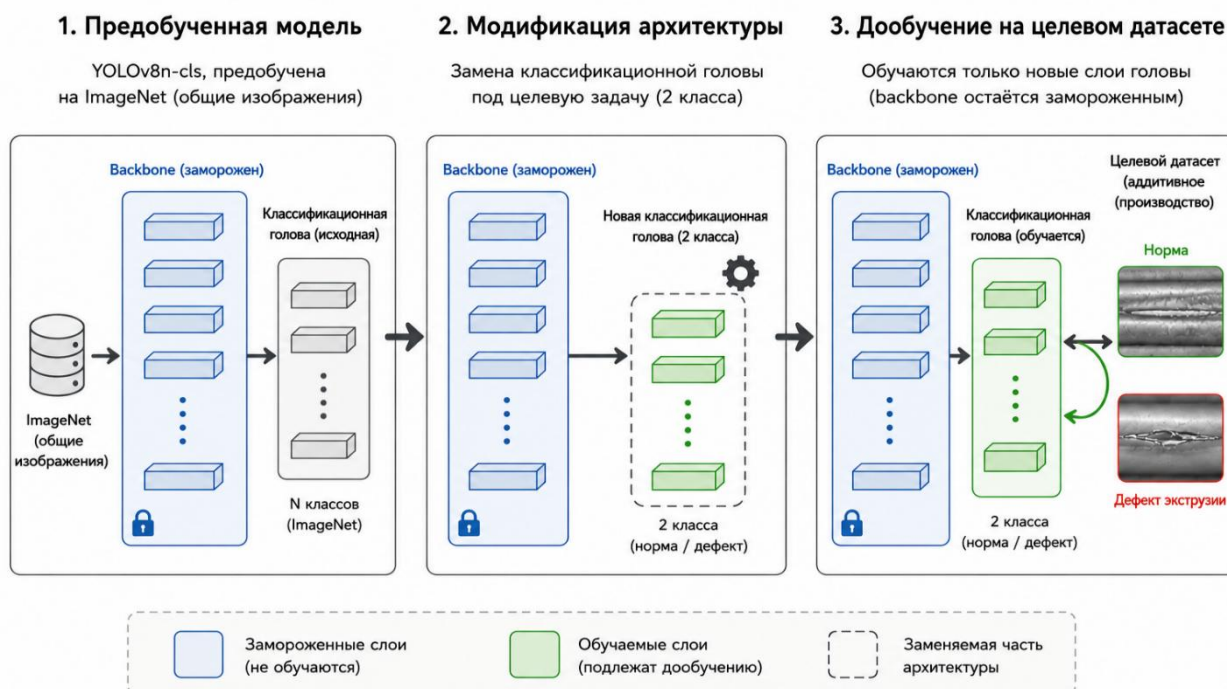


Рисунок 1.3 – Схема процесса transfer learning при адаптации YOLOv8n-cls

Модель содержит всего 2,7 миллиона параметров, вследствие этого обеспечивается малая вычислительная нагрузка [1]. Оперативная память расходуется экономно, промежуточные тензоры не перегружают систему, а графический интерфейс остается отзывчивым даже при непрерывной обработке видеопотока. Так как цветовая информация не несет полезных признаков для

классификации, было принято решение подавать на вход монохромное изображение. Это предупреждает возможность введение модели в заблуждение, поскольку нейросеть способна выявить ложные корреляции, например, на основании факта, что в выборке датасета с дефектом изображений с зеленым пластиком больше остальных, можно сделать ошибочный вывод «зеленый пластик = дефект». Подобные артефакты обучения снижают практическую ценность модели.

Техническая реализация сохраняет совместимость с предобученной архитектурой. Ожидаемое на первом сверточном слое YOLOv8n-cls трехканальное изображение формируется путем простого дублирования серого канала в три идентичных плоскости, формируя псевдо-RGB тензор. Такое решение сочетает архитектурную совместимость и семантическую устойчивость.

1.4 Теория управления по последовательному интерфейсу и командному языку G-код

Физическое подключение принтера к компьютеру происходит посредством USB интерфейса, операционная система эмулирует виртуальный COM-порт для обмена пакетами данных. Внутри материнской платы сигнал преобразуется в асинхронный UART, а микросхемы наподобие CH340 обеспечивает трансляцию необходимых уровней напряжения. Все это взаимодействие, реализуемое в программе, осуществляется библиотекой pyserial [2].

Управление самим технологическим процессом для настольных FDM-систем выполняется через использование набора команд на языке G-код. Каждая строка состоит из адресного слова и набора параметров. Примеры основных команд G-кода для управления FDM-принтером приведены в таблице 1.8. Префиксы делятся на две группы. Первая – геометрические команды G0/G1, управляющие линейным перемещением осей и экструзией. Вторая – вспомогательные команды M0/M599, способны регулировать температуру нагревателей, управляют вентиляторами обдува, запрашивают текущие координаты, но

реальный функционал M-команд значительно шире и включает сотни других операций. Протокол обмена опирается на строгую последовательность «запрос–подтверждение», в то время пока хост отправляет инструкцию, микроконтроллер ставит ее в очередь, возвращает строку ok.

Таблица 1.8 – Основные команды G-кода для управления FDM-принтером

Команда	Описание	Параметры	Пример использования
G0	Быстрое перемещение без экструзии	X, Y, Z, F (координаты и скорость)	G0 X100 Y50 F3000
G1	Линейное перемещение с экструзией	X, Y, Z, E, F	G1 X10 Y20 E0.5 F1500
G28	Возврат в нулевую точку (home)	X, Y, Z (опционально)	G28 X Y
G90	Абсолютная система координат	—	G90
G91	Относительная система координат	—	G91
M25	Приостановка SD-печати	—	M25
M104	Установка температуры экструдера	S (температура °C)	M104 S200
M140	Установка температуры стола	S (температура °C)	M140 S60
M105	Запрос текущей температуры	—	M105
M84	Отключение шаговых двигателей	E (опционально)	M84
M112	Аварийное отключение (Emergency Stop)	—	M112

Примечание: Полный синтаксис команд зависит от версии прошивки (Marlin, Klipper, Repetier).

Интерпретатор команд и аппаратные лимиты определяются прошивкой микроконтроллера. Выбор семейства Marlin [3] для данного проекта определен следующими факторами. Имеющееся оборудование – принтер Anycubic Kobra 2 Neo – использует именно эту прошивку. Второе, Marlin доминирует в сегменте настольных FDM-принтеров, среди таких устройств доля распространенности достигает 70–80%. Основная необходимость столь детального

определения технических параметров 3D-принтера вытекает из задания прерывания процесса печати. Команда M112 является стандартным способом аварийной остановки, она обесточивает двигатели и отключает нагреватели. Ключевой же недостаток такой команды, после ее выполнения устройство полностью теряет связь с управляющей программой и уходит в защиту, фиксируя превышение напряжения. Именно поэтому в проекте выбор сделан в пользу команды M25. Она ставит процесс на паузу, запоминает текущую позицию и оставляет контроллер в активном состоянии. Это позволяет возобновить печать с места остановки, что является критически важным, особенно при неправильно выбранном пользователем пороге срабатывания.

Понимание физических лимитов и семантики протокола составило ядро для построения надежного контура управления. Локальная обработка исключает облачные задержки, протокол гарантирует целостность команд, а выбор программной паузы обеспечивает безопасность и возможность возобновления печати. Эти принципы легли в основу модуля управления, описанного в третьей главе.

Выводы по первой главе

При сравнении существующих решений в области автоматизированного контроля качества аддитивного производства было выявлено несоответствие между потребностями в гибкости и автономности реальных производств и ограничениями применяемых средств. Дальнейшее проектирование было продиктовано необходимостью соблюдения выдвинутых требований.

Анализ архитектур нейросетевых моделей и проведение экспериментальной оценки вычислительной нагрузки позволили выбрать YOLOv8n-cls среди рассмотренных вариантов. В условиях ограниченных ресурсов ключевым стало сочетание трансферного обучения и предобработки кадров. Именно оно позволило сохранить чувствительность к дефектам без потери производи-

тельности. В части аппаратного взаимодействия рассмотрели особенности последовательного протокола UART, семантику команд G-кода для прошивки Marlin.

Изложенные в первой главе теоретические и методологические основы определяют направление последующего проектирования. На требованиях, выбранной архитектуре и протоколах строится практическая реализация.

2 РАЗРАБОТКА И ВАЛИДАЦИЯ МОДЕЛИ МАШИННОГО ЗРЕНИЯ

2.1 Формирование, предобработка и аугментация датасета

Эффективность обучения модели напрямую зависит от качества, сбалансированности и размера датасета. Обучающая выборка для задачи детекции дефектов экструзии была сформирована по следующему принципу, изображения были разделены на два класса: `no_error_print` (штатный режим печати) и `extrusion_error_print` (наличие визуальных артефактов). Источниками данных послужили открытые репозитории компьютерного зрения для аддитивного производства [4], [5].

С помощью первоначального анализа была выявлена необходимость адаптации обучающих данных. «Сырые» изображения различных открытых источников, объединенные в единый датасет различались по множеству параметров, например, разрешению, масштабу или формату. Использование их без предварительной обработки не позволяло модели корректно обобщать признаки, вместо обнаружения дефектов происходило переобучение на нерелевантных признаках, что, в свою очередь снижало робастность детекции. Для устранения данных эффектов был разработан пайплайн, состоящий из следующих этапов:

1. Центральный квадратный кроп (Central Square Cropping). Из исходного кадра выделяется центральная квадратная область. Ключевая идея необходимости данной операции заключается в попытке обеспечить единое соотношение сторон, которое сделает возможным применение модели с камерами, имеющими различное исходное соотношение сторон (например, 16:9, 4:3, 3:2).
2. Конвертация в оттенки серого (Grayscale Conversion). Дефекты FDM-печати носят структурный и текстурный характер (этот вопрос частично рассматривался ранее), тогда как цвет пластика варьируется в зависимости от

материала и партии, а перевод в монохромный формат исключает ложные корреляции.

3. Дублирование серого канала в RGB-формат. Архитектура YOLOv8n-cls предполагает трехканальный вход, поэтому, чтобы не модифицировать модель, одноканальное изображение в оттенках серого преобразуется в трехканальное путем тривиального дублирования одного и того же массива значений интенсивности во все три канала ($R = G = B$). Такой подход позволяет использовать предобученные веса (трансферное обучение) при сохранении фокуса модели исключительно на текстурных признаках.

Для расширения вариативности выборки и повышения устойчивости к изменениям условий использования алгоритма применялась аугментация данных. Этот подход обеспечивает увеличение объема обучающих данных за счет создания модифицированных версий изображений при сохранении целевых признаков, необходимых для тренировки модели. Среди использованных методов аугментации можно выделить:

1. Случайная обрезка с масштабированием. Увеличение масштаба до 10% с случайным кропом, этот прием компенсирует естественные вариации расстояния между камерой и зоной печати и предотвращает переобучение модели на конкретной геометрии кадра.

2. Поворот. Случайный угол вращения относительно центра кадра в диапазоне $\pm 15^\circ$, имитируя возможное смещение угла обзора, повышает инвариантность модели.

3. Сдвиг. Горизонтальное и вертикальное смещение кадра в пределах $\pm 10\%$ от исходных размеров моделирует смещение зоны формирования изделия, что является особенно актуально для FDM-принтеров с подвижной платформой.

4. Коррекция экспозиции. Изменение яркости изображения в пределах $\pm 5\%$ позволяет обучить модель с имитацией изменчивости внешнего освещения, характерного для производственных помещений.

5. Добавление шума. Случайное искажение до 0.46% пикселей изображения имитирует артефакты сжатия видеопотока и матричный шум камеры, возникающий в условиях пониженной освещенности.

Эмпирически были подобраны параметры аугментации, чрезмерно агрессивные трансформации (повороты более 20°, сильное искажение экспозиции) приводили к снижению точности модели на реальных данных. Обработка и создание различных версий датасета осуществлялись на платформе Roboflow [6], предоставляющей готовый конвейер трансформаций и автоматическое соблюдение пропорций и экспорт в формате, совместимом с библиотекой ultralytics.

Итоговый объем датасета составил 46664 изображения, что примерно в три раза больше исходной выборки. После аугментации данные были сформированы три непересекающиеся подвыборки (рисунок 2.1).

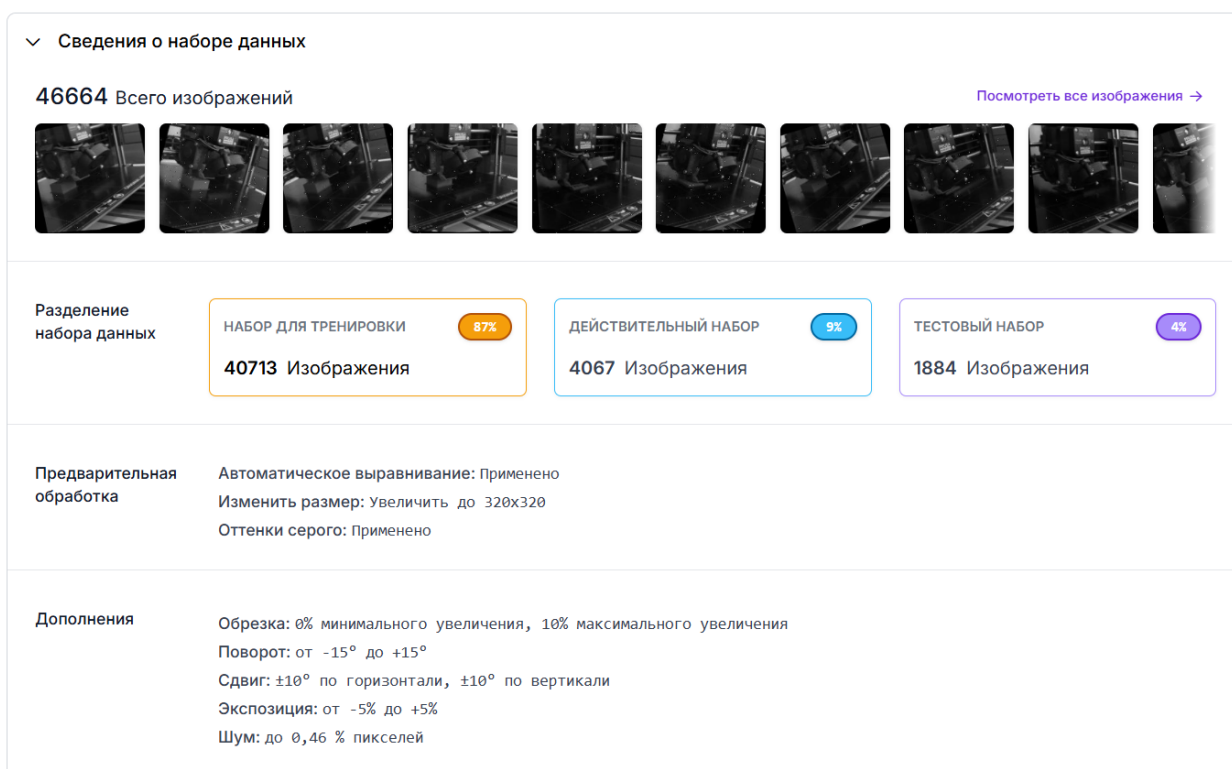


Рисунок 2.1 – Структура датасета и разбиение на выборки в платформе Roboflow

Подобное распределение (87/9/4) является стандартным для задач бинарной классификации. Доминирующая доля данных отводится обучению, оставшаяся доля – валидационная и тестовая. Для исключения ассиметрии

классы в каждой подвыборке балансируются автоматически. Сформированный и аугментированный датасет лег в основу эксперимента по обучению модели. Детали этого эксперимента приведены в следующем подразделе.

2.2 Методика обучения и оптимизация гиперпараметров

В качестве базовой архитектуры была выбрана yolov8n-cls.pt – компактная предварительно обученная модель семейства YOLOv8, содержащая примерно 2.8 млн параметров. Обучение модели осуществлялось в среде глубокого обучения PyTorch с использованием официального фреймворка ultralytics [7]. До запуска обучения требуется задать набор гиперпараметров, определяющих его ход. Основные из них:

- Epochs (количество эпох). Количество полных проходов всей обучающей выборки через модель в процессе обучения нейронной сети. Мониторинг точности валидации помогает определить оптимальное значение.
- Imgsz (размер изображений). Масштабирование изображений, загружаемых в модель с сохранением исходного соотношения сторон, оставшееся пространство заполняется нулевыми (черными) пикселями.
- Batch (размер батча). Количество обучающих изображений, которые обрабатываются моделью за один шаг перед обновлением ее весов.
- Workers (рабочие процессы или потоки). Параметр, управляющий параллельной загрузкой данных во время обучения нейронной сети.

Корректное выставление данных параметров позволяет быстро и эффективно обучать модель согласно имеющимся ресурсам. Анализируя производительность в диспетчере задач, можно эмпирически подобрать эти значения. Нагрузка на RAM, CPU и GPU свыше 85% является критической. В результате практических экспериментов было выявлено:

- Batch нагружает видеокарту (GPU) и расходует VRAM.
- Workers нагружает процессор (CPU) и расходует RAM.

- Значение `imgsz` следует выбирать таким, чтобы оставались различимы признаки необходимые для классификации.
- Значение `epochs` выбирается в соответствии с показателями `loss` и `top1_acc`, выводящимися в логах при обучении модели.

Минимизация времени обучения и возможности используемого оборудования являлись основными факторами, с учетом которых подбирались гиперпараметры. Итоговые значения приведены в таблице 2.1.

Таблица 2.1 – Гиперпараметры обучения и обоснование выбора

Параметр	Значение	Обоснование
<code>model</code>	<code>yolov8n-cls.pt</code>	Компактная архитектура (~2.8 млн параметров) с предобучением на ImageNet для ускоренной сходимости.
<code>imgsz</code>	320	Стандартный размер для классификации; баланс детализации и нагрузки на VRAM.
<code>batch</code>	512	Максимальная загрузка GPU при малом разрешении входа; стабилизация градиентов.
<code>epochs</code>	20	Стабилизация метрик после 10–15 эпохи; обучение более 20 эпох нецелесообразно.
<code>workers</code>	1	Стабильная работа DataLoader в Windows без конфликтов потоков и избыточного потребления RAM.
<code>device</code>	0	Принудительное использование GPU (CUDA) для ускорения обучения.

Контроль процесса происходил путем мониторинга четырех ключевых метрик, функции потерь на обучающей выборке (`train/cls_loss`), функции потерь на валидационной выборке (`val/cls_loss`), метрики точности топ-1 (`metrics/accuracy_top1`), точности топ-5 (`metrics/accuracy_top5`). Динамика изменения данных метрик по эпохам представлена на рисунке 2.2.

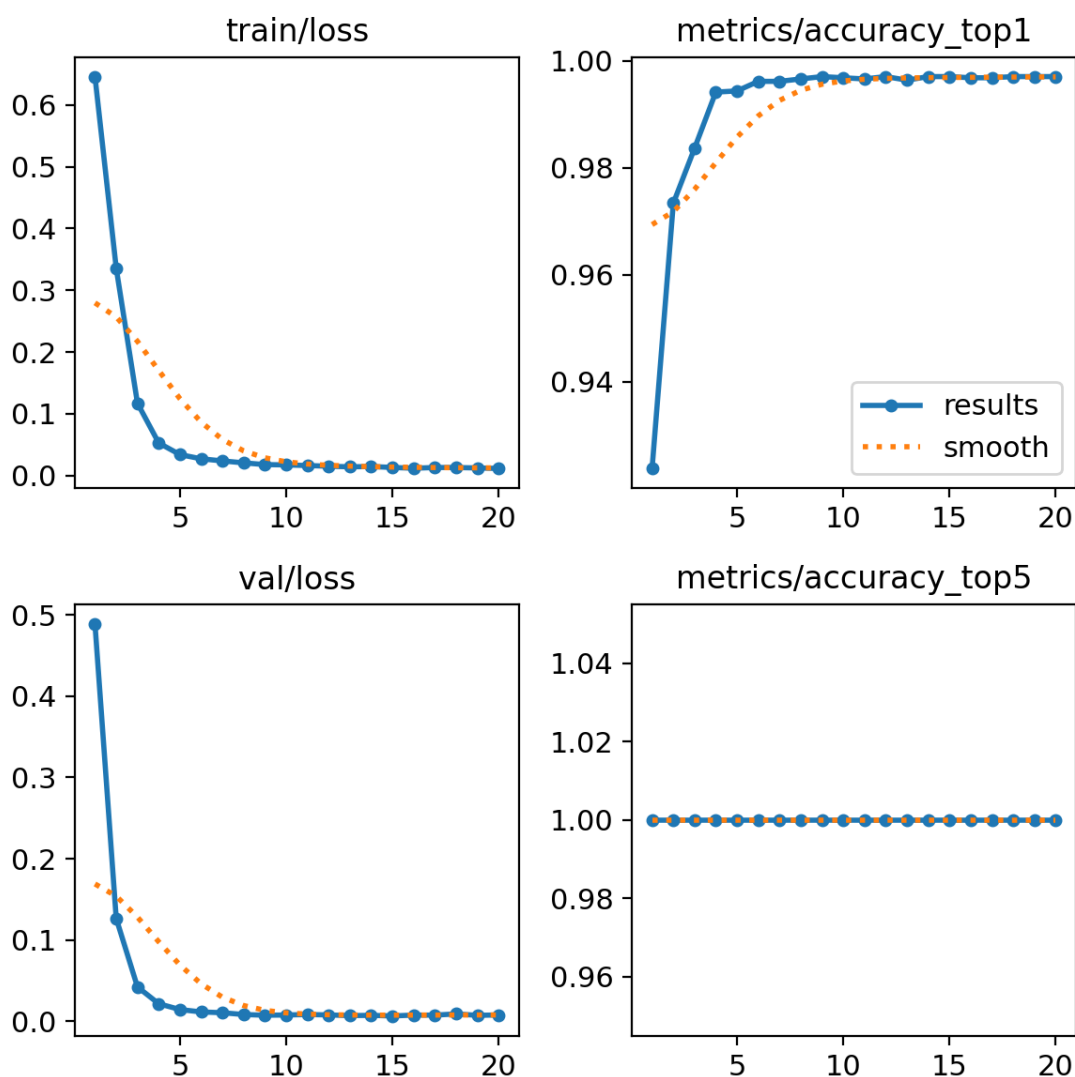


Рисунок 2.2 – Динамика изменения функции потерь и точности по эпохам обучения

Анализ кривых и численных значений демонстрирует высокую сходимость модели. Финальная точность на 20-й эпохе составила 99.71% при валидационной потере 0.00782. Разрыв между train/loss и val/loss к концу обучения – менее 0.004 (относительная разница $\sim 0.5\%$), что подтверждает отсутствие переобучения. Для получения конкретных численных значений на каждой эпохе, оценки времени обучения, целесообразно обратиться к таблице 2.2.

Из табличных данных следует, что пик динамики обучения – снижение функции потерь и повышение точности – приходится на первые пять эпох. Уже к 10-й эпохе наблюдается выход метрик на плато, что указывает на завершение активной сходимости.

Таблица 2.2 – Ключевые показатели динамики обучения модели

epoch	train/loss	metrics/accuracy_top1	val/loss
1	0.64608	0.9238	0.48848
2	0.33572	0.97341	0.12644
3	0.11705	0.98369	0.04227
4	0.05277	0.99419	0.0216
5	0.03387	0.99441	0.01443
6	0.02705	0.9962	0.0115
7	0.02364	0.9962	0.01052
8	0.02051	0.99665	0.00832
9	0.01769	0.99709	0.00746
10	0.01718	0.99687	0.00773
11	0.01616	0.99665	0.00831
12	0.01504	0.99709	0.00782
13	0.01406	0.99642	0.00696
14	0.01455	0.99709	0.00732
15	0.01307	0.99709	0.00652
16	0.01178	0.99687	0.00765
17	0.01296	0.99687	0.00755
18	0.01295	0.99709	0.00909
19	0.01224	0.99709	0.00729
20	0.01141	0.99709	0.00782

Пятнадцатая эпоха оказалась ключевой: именно на ней достигнуты минимальное значение валидационной потери ($\text{val/loss} = 0,00652$) и максимальная точность ($\text{accuracy_top1} = 0,99709$). Достигнутый показатель точности объясняется совокупностью технологических и методологических факторов:

1. Бинарный характер задачи. Так как в сформированном датасете разделение происходит всего на два визуально контрастных класса («отсутствие аномалий» / «выраженный дефект экструзии») задача радикально упрощается по сравнению с мультиклассовой классификацией.

2. Трансферное обучение. Использование хорошо предобученной модели и отсутствие необходимости построения архитектуры нейронной сети существенно облегчило задачу. Дообучение на 46 664 изображениях обеспечило быструю адаптацию без деградации весов.

3. Предобработка данных. Как уже было отмечено ранее, конвертация в оттенки серого и другие преобразования помогают предупредить ложные корреляции.

4. Отсутствие переобучения. Разрыв между показателями train/loss и val/loss не превышает 0,5%, что свидетельствует об отсутствии каких-либо признаков переобучения.

Таким образом, произведенное дообучение модели с эмпирически подобранными параметрами позволило достичь целевых показателей точности и устойчивости сформулированных в требованиях ранее.

2.3 Валидация модели и оценка производительности

Для оценки качества классификации использовались стандартные метрики: точность (Accuracy), точность класса «дефект» (Precision) Расчет осуществлялся на основе матрицы ошибок (Confusion Matrix), значения из которой, для удобства, сведены в таблицу 2.3.

Таблица 2.3 – Матрица ошибок на тестовой выборке

Предсказано \ Фактически	no_error_print	extrusion_error_print
no_error_print	938 (TN)	3 (FN)
extrusion_error_print	2 (FP)	941 (TP)

Расшифровка обозначений матрицы ошибок:

- **TP** (True Positive) – истинно положительные срабатывания: модель корректно идентифицировала изображение с дефектом экструзии;
- **TN** (True Negative) – истинно отрицательные срабатывания: модель корректно идентифицировала изображение штатного режима печати;
- **FP** (False Positive) – ложно положительные срабатывания: модель ошибочно классифицировала нормальное изображение как дефектное;
- **FN** (False Negative) – ложно отрицательные срабатывания: модель пропустила реальное изображение с дефектом.

На основе данных матрицы рассчитаны итоговые показатели:

$$Precision = \frac{TP}{TP + FP} = \frac{941}{941 + 2} = 0.9979$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{941 + 938}{1884} = 0.9973$$

Полученные значения удовлетворяют формализованным требованиям из таблицы 1.2. Требование TP-1 ($Accuracy \geq 90\%$) выполнено с запасом – достигнуто 99,71%. Требование TP-2 ($Precision \text{ класса «дефект»} \geq 0,85$) также превышено: итоговое значение составило 99,82%. Обеспечение требований TP-3 и TP-4 выполнено на этапе разработки. Устойчивость к изменению освещения (TP-3) обеспечивается на уровне предобработки (конвертация в оттенки серого) и аугментации данных (вариации экспозиции $\pm 5\%$). Низкий уровень ложных срабатываний (TP-4) достигается за счет высокого порога уверенности (по умолчанию 0.8) – его настройка детально рассмотрена в разделе, посвященном программной реализации.

Для оценки пригодности модели к работе в замкнутом контуре управления был проведен анализ временных затрат и потребления вычислительных ресурсов. Согласно требованиям из таблицы 1.3, система должна обеспечить не менее 7 кадров/с и загрузку процессора не более 60%. Для инференса модели использовалась целевая вычислительная платформа – AMD Ryzen 5

5600U, 32 ГБ ОЗУ и GPU NVIDIA RTX 4060. Полученные характеристики производительности сведены в таблицу 2.4.

Таблица 2.4 – Характеристики производительности модели

Показатель	Значение
Время инференса (GPU)	12–18 мс
Время инференса (CPU)	45–65 мс
Частота анализа кадров	7.5–8.0 FPS
Загрузка CPU (система в целом)	40–55%
Потребление RAM	280–320 МБ
Общая задержка пайплайна (от камеры до команды)	150–250 мс

Все показатели находятся в пределах ранее определенных допустимых значений. Частота кадров и нагрузка процессора соответствуют требованиям проектирования. Таким образом, предложенный подход к формированию датасета, обучению и валидации модели доказал свою эффективность.

Выводы по второй главе

В рамках раздела были выполнены работы по дообучению и тестированию модели YOLOv8n-cls для поиска дефектов при 3D-печати. Датасет сформировали путем сбора данных из открытых источников, предобработка позволила привести все изображения к единому формату, а аугментация - искусственно расширить обучающую выборку.

К 15-й эпохе модель вышла на 99,71 % точности, а ошибка на проверочных данных зафиксировалась на уровне 0,00652. Разрыв между обучающей и валидационной функциями потерь не превысил 0,5 %, что подтверждает отсутствие переобучения. Оценка производительности и точности инференса удовлетворяет выдвинутым ранее требованиям. Разработанная нейросетевая модель полностью соответствует техническим требованиям и готова к интеграции в программно-аппаратный контур управления 3D-принтером.

3. ПРОГРАММНО-АППАРАТНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ АВТОМАТИЧЕСКОГО УПРАВЛЕНИЯ

3.1 Архитектура системы и организация контура управления

Программная система построена как гибридная событийно-ориентированная среда. В ней объединены модуль компьютерного зрения, алгоритмы принятия решений и интерфейс для управления 3D-принтером. Что важно, архитектура предполагает четкое разделение потоков исполнения. Это дает сразу несколько плюсов: интерфейс остается отзывчивым, область анализа можно гибко настраивать, а состояние печати – безопасно контролировать.

Логика работы системы реализует контр автоматического контроля с явным управлением состояниями (рисунок 3.1). В рамках контура реализован полный цикл: от получения визуальных данных с камеры до выдачи управляющей команды на принтер.

Ключевые особенности реализации контура:

1. Гибкая настройка области анализа. Оператор может использовать увеличение для корректировки визуальных данных, входящих в модель. Действие применяется до инференса, это позволяет отсеять информацию не содержащую целевых признаков.
2. Строгая фильтрация и защита от спама. Реакция активируется только при одновременном выполнении условий: модель распознает дефект и относит его к соответствующему классу и коэффициент уверенности модели больше или равен выставленному оператором порогу.
3. Синхронизация с внешними событиями. Метод `poll_serial()` анализирует ответы микроконтроллера принтера, при обнаружении контрольной суммы кнопка возобновления печати деактивируется, логируется переход в режим мониторинга.

4. Управляемое возобновление. После остановки по команде M25 система переходит в состояние `print_paused`, активируется кнопка возобновления печати, ожидается подтверждение оператора. Отправка M24 возобновляет процесс печати, сбрасывается флаг, исключая несанкционированное продолжение печати.



Рисунок 3.1 – Функциональная блок-схема системы мониторинга и управления

Чтобы воплотить описанный контур на практике, система была разбита на три основных класса. Каждый из них отвечает за свою функциональную область.

Класс `CameraThread` (наследник `QThread`) – фоновый поток обработки видеоданных. Выполняет:

- Захват кадров с камеры (`cv2.VideoCapture`) [8];
- Предобработку: конвертацию в оттенки серого и дублирование канала для совместимости с входом `YOLOv8n-cls`;
- Инференс модели с параметром `imgsz=320` и отключенным выводом (`verbose=False`);

- Пропуск каждого второго кадра для снижения нагрузки на центральный процессор;
- Визуализацию зеленой рамки ROI на отображаемом кадре для обратной связи с оператором;
- Формирование сигнала с копией кадра, классом предсказания и коэффициентом уверенности.

Класс `PrinterSerial` – модуль взаимодействия с микроконтроллером принтера по последовательному интерфейсу. Реализует:

- Автоматический поиск порта;
- Инициализацию соединения с заданными параметрами;
- Очистку входного буфера перед отправкой команды для предотвращения накопления «мусорных» данных;
- Методы `send_cmd()` и `read_all()` для асинхронного обмена командами и чтения ответов.

Класс `MainWindow` – главный поток приложения, отвечающий за:

- Отрисовку графического интерфейса на базе PyQt6 [9]
- Прием сигналов от `CameraThread` и обновление виджета видео;
- Управление состоянием печати через флаг `print_paused` и кнопку «Возобновить печать (M24)»;
- Обработку пользовательских событий: изменение порога срабатывания, настройка цифрового зума, выбор режима реакции;
- Периодический опрос COM-порта через `QTimer` с интервалом 100 мс с автоматическим сбросом флага паузы при обнаружении внешнего сигнала возобновления;
- Централизованное логирование событий в файл и консоль интерфейса.

Интерфейс разделен на две функциональные зоны (рисунок 3.2), что позволяет эргономично разнести элементы управления и информационные

поля. Левая панель фиксированного размера. На ней расположены: список режимов реакции, ползунки порога уверенности и цифрового зума, индикатор статуса, кнопки переключения, возобновления печати. Правая часть является расширяемой и отведена под видеопоток с текстовым оверлеем и консоль событий.

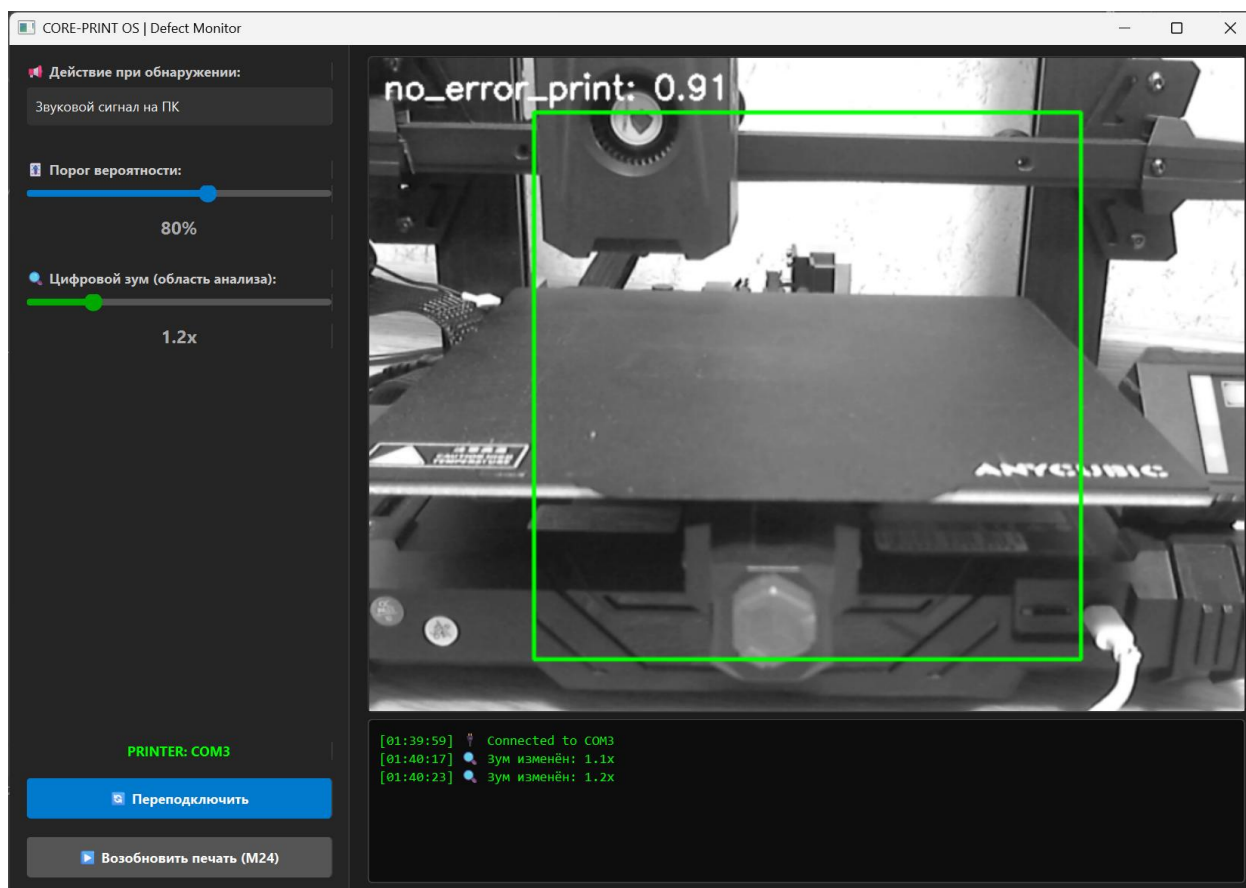


Рисунок 3.2 – Графический интерфейс системы

Совершенные архитектурные решения были обусловлены объективными факторами, позволяющими предупредить ошибки, обеспечить безопасность и улучшить пользовательский опыт. Двойное логирование и в графическом интерфейсе и в файл позволяет восстановить ход выполнения программы в случае необходимости. Ручное подтверждение возобновления печати предотвращает случайный запуск принтера при внештатных ситуациях, изоляция потоков обеспечивает необходимый уровень отказоустойчивости, также ряд принятых решений был нацелен на исключение блокировки интерфейса при изменении каких-либо настроек пользователем или инференсе модели.

Таким образом, разработанная архитектура гарантирует надежную работу системы в реальном времени, готовность к интеграции в производственный контур управления с поддержкой ручного вмешательства и гибкой настройкой области анализа без потери точности классификации.

3.2 Программная реализация последовательного интерфейса USB/UART

Обмен данными между системой визуального контроля и микроконтроллером 3D-принтера организован посредством асинхронного последовательного соединения, работающего через интерфейс USB. В рассматриваемой схеме переключатель передает сигнал на аппаратный преобразователь. После этого преобразователь создает виртуальный COM-порт. Через него система получает необходимые данные и передает команды оборудованию.

Для программной реализации канала связи используется библиотека `pyserial` – она содержит весь необходимый функционал для работы с последовательным портом. Чтобы добиться минимальной задержки, применялся неблокирующий режим чтения с таймаутом 0,1 с и скоростью обмена 115200 бит/с. Эти настройки – стандарт для прошивок Marlin. Их пропускной способности хватает для команд G-кода, а буфер при этом не переполняется.

Удобство эксплуатации повысилось за счет разработанного алгоритма автоматической идентификации устройства. Он избавляет пользователя от необходимости вручную выбирать COM-порт. Сканирование всех доступных последовательных интерфейсов осуществляется через специальную утилиту `serial.tools.list_ports.comports()`, после производилась фильтрация результатов производится по драйверам, характерным для контроллеров FDM-оборудования (CH340, FTDI, CP210x, CDC-ACM), что позволяет системе безошибочно находить целевой порт среди прочих USB-устройств.

В основе управления буферами и отправки команд – реальные условия эксплуатации. Принтер в процессе работы накапливает команды во входном буфере. Если связь прерывается или программный контур зависает, буфер

начинает копить устаревшие инструкции. Чтобы избежать наложения команд, перед каждой отправкой вызывается `reset_input_buffer()` – он очищает очередь приема микроконтроллера.

Сама команда – это ASCII-строка с символом новой строки (`\n`) в конце, затем она кодируется в байты. Метод `send_cmd()` возвращает логический статус: успешна ли запись. Так модуль принятия решений точно знает, дошли ли критические инструкции (M25, M24).

Чтение данных из порта – неблокирующее, работает через проверку атрибута `in_waiting`. При появлении данных во входном буфере выполняется их декодирование в строковый формат. Ошибки кодировки при чтении преднамеренно игнорируются (`errors='ignore'`), после чего полученная информация разделяется на отдельные строки для дальнейшей обработки. Благодаря этому поток продолжает работать стабильно даже в тех случаях, когда принтер временно не передает ответ. Работа интерфейса и обработка видеопотока при этом не прерываются.

Контроль входящих сообщений реализован через `QTimer`, выполняющий опрос микроконтроллера с интервалом 100 мс. Они попадают в системную консоль, логируются, и система анализирует – не изменилось ли состояние печати.

Если кабель отключили, драйвер дал сбой или переполнился буфер – всплывает исключение `serial.SerialException`. Модуль ловит ошибку, закрывает дескриптор порта (`ser.close()`) и переводит индикатор статуса в `OFFLINE`. Пользователь жмет «Переподключить» – система заново сканирует порты, запускает рукопожатие. Необходимость перезапуска приложения отпадает.

За счет выбранного способа обмена данными связь между модулями сохраняет устойчивость даже при нестабильной работе оборудования. Это особенно важно для непрерывного контроля процесса печати на начальном этапе производства.

3.3 Интеграция программных модулей и проверка устойчивости системы

После того как каждый модуль был разработан и отлажен отдельно, последовал этап сборки. Процесс печати зачастую является довольно длительным процессом, поэтому система также должна быть стабильной.

В программный комплекс входят несколько независимых частей, а именно: модуль захвата видеопотока, нейросетевой блок, интерфейс оператора и подсистема для связи с микроконтроллером принтера. Элементы работают независимо, что формирует распределение нагрузки по потокам и исключает блокировку интерфейса.

Камера снимает непрерывно, каждый кадр перед отправкой в модель YOLOv8n-cls проходит предварительную обработку. Результат классификации возвращается в главный поток, где проверяется уровень уверенности модели. Как только порог превышен, система формирует команду остановки печати. Реакция системы на обнаружение дефекта экструзии с автоматической отправкой команды M25 показана на рисунке 3.3.

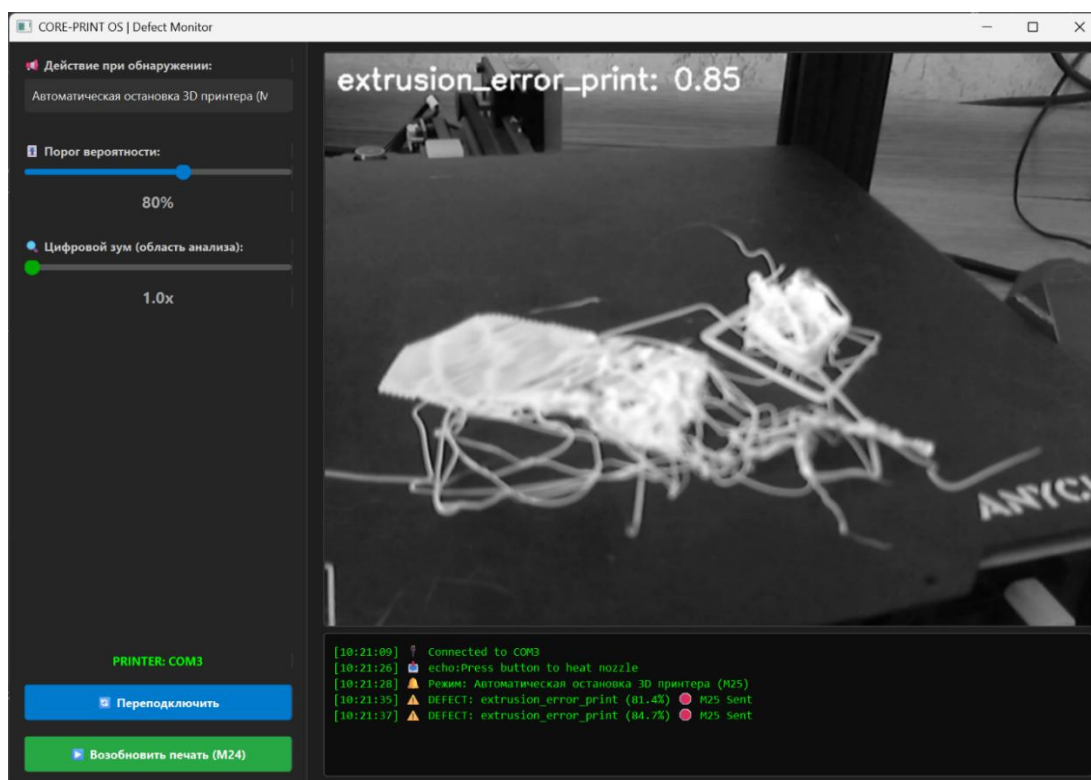


Рисунок 3.3 – Реакция системы на обнаружение дефекта экструзии

При разработке особое внимание уделили устойчивости интерфейса. В первых версиях инференс запускался в основном потоке – приложение начинало тормозить, особенно при долгой работе. Тогда обработку видеопотока и общение с моделью вынесли в отдельный поток. Результат: графический интерфейс остается отзывчивым даже при непрерывном анализе кадров.

Дополнительно реализовали систему журналирования событий. В лог попадают сведения о подключении оборудования, отправке команд, обнаружении дефектов и ошибках. Это помогает разбираться в поведении системы и находить причину остановки печати в нештатных ситуациях.

В ходе тестирования отдельно проверяли поведение программы при обрыве связи с принтером. Отключение USB-кабеля, переполнение буфера, ни сбой драйвера не приводили к аварийному останову программы. Реакция интерфейса на обрыв связи с принтером показана на рисунке 3.4.

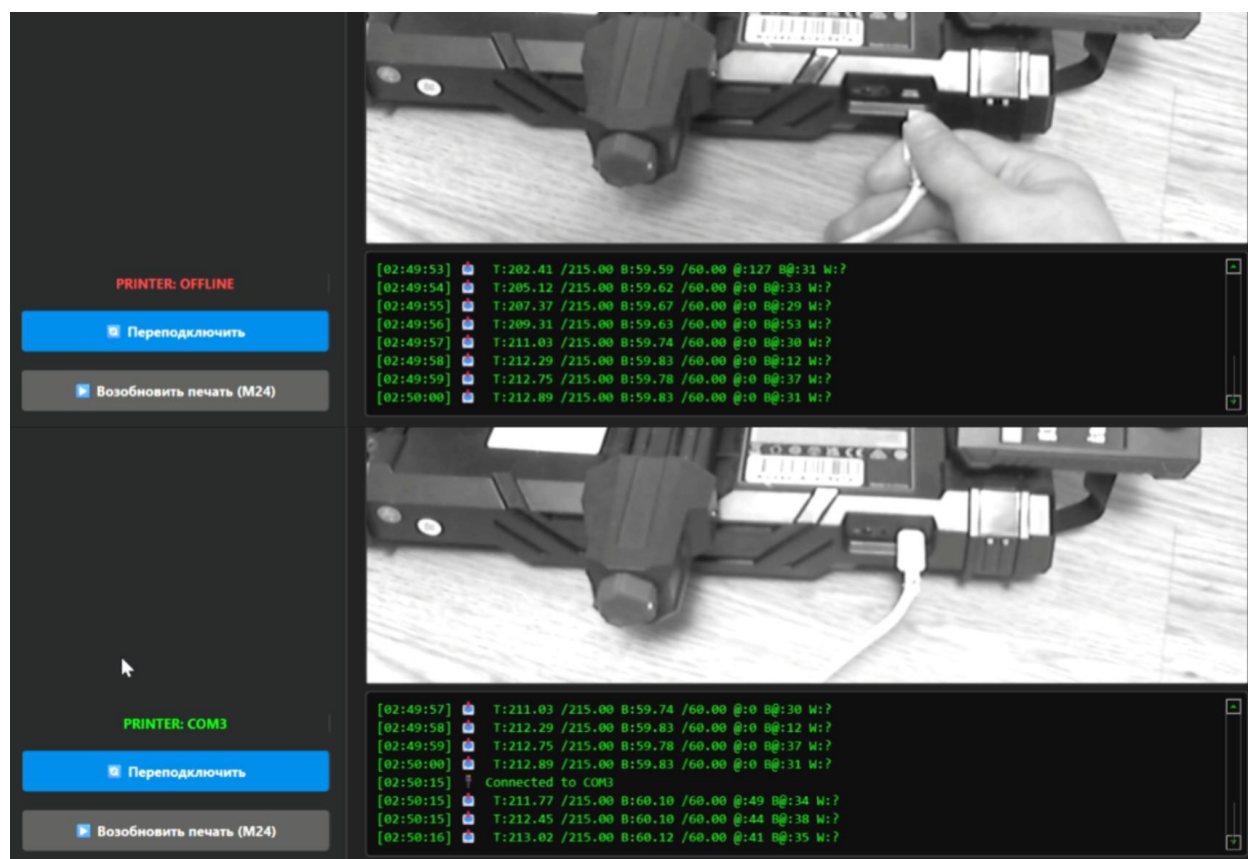


Рисунок 3.4 – Реакция системы на обрыв связи с принтером

При потере соединения система отображает этот факт в графическом интерфейсе, переходит в режим ожидания и остается в этом состоянии до повторного подключения устройства. После восстановления связи система заново производит сканирование COM-портов и запускает инициализацию.

Также оценивали стабильность при длительной работе. При непрерывном тестировании контролировали загрузку процессора, потребление ОЗУ и частоту обработки кадров. Мониторинг ресурсов системы в диспетчере задач (рисунок 3.5) отображает, что на процессоре AMD Ryzen 5 5600U нагрузка стабильно держалась в пределах 40–55%. Этого достаточно, чтобы одновременно работали интерфейс, видеопоток и обмен данными с принтером. При оптимально настроенном пороге чувствительности ложные срабатывания не наблюдались, а отключение принтера, в свою очередь, не нарушало работоспособности системы.

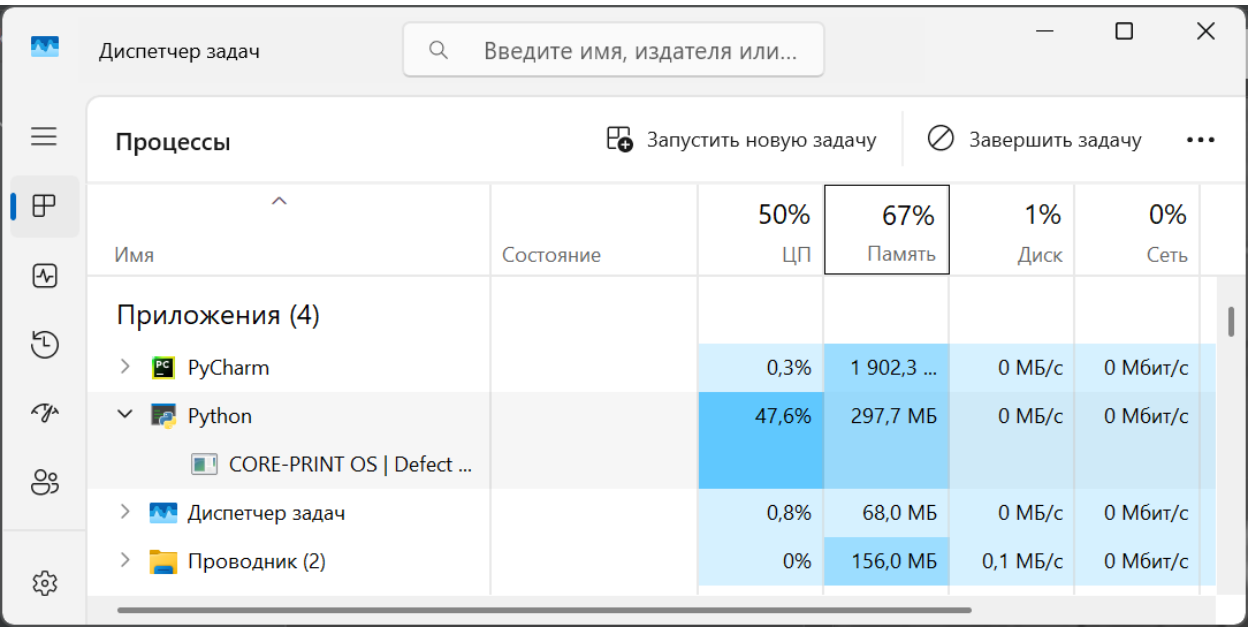


Рисунок 3.5 - Мониторинг ресурсов системы в диспетчере задач

Практические испытания подтвердили работоспособность системы. Она способна непрерывно следить за печатью в реальном времени. Заметных задержек интерфейса и потери связи с оборудованием при этом не возникает. Устойчивая работа комплекса при длительной эксплуатации – прямой результат интеграции модулей в единую архитектуру.

Выводы по третьей главе

В третьей главе представлена работоспособная программно-аппаратная архитектура с разделением потоков выполнения. Захват видеопотока и инференс нейросети вынесены в отдельный поток исполнения – CameraThread. Такое архитектурное решение сохраняет отзывчивость интерфейса в процессе длительной эксплуатации. Зависания системы при этом полностью исключаются. В основе управления лежит конечный автомат (State Machine), реализующий событийно-ориентированную логику. При обнаружении дефекта система переводит принтер в состояние «Пауза». Для возобновления работы требуется явное подтверждение оператора. Указанный подход исключает случайные запуски.

Реализованы механизмы повышения отказоустойчивости. Например, модуль pyserial, отвечающий за связь с принтером, способен восстанавливать соединение при его потере, блокировка спама командами и автоматическая остановка через 2 минуты бездействия оператора. Интеграционные задачи решены полностью, при тестировании на штатном оборудовании система продемонстрировала стабильную работу и низкое потребление ресурсов.

4 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ

Целью данного дипломного проекта является разработка системы автоматического обнаружения дефектов при 3D-печати на основе нейросетевой архитектуры YOLOv8.

В работе был сформирован и размечен датасет (база для обучения нейросети) путем объединения уже существующих аналогов из открытых источников, проведена работа по реализации алгоритма для предобработки входящих изображений и расширению датасета с помощью аугментации данных, произведено обучение и тонкая настройка модели YOLOv8, а также разработаны программный модуль по управлению параметрами печати 3D-принтера через последовательный порт и графический пользовательский интерфейс (GUI) приложения.

Для практической апробации и тестирования использовался настольный FDM-принтер Anycubic Kobra 2 Neo в качестве объекта управления и веб-камера Logitech C310 для захвата видеопотока зоны печати.

4.1 Расчет затрат на оплату труда

Для определения стоимости разработки данного проекта необходимо рассчитать трудоемкость выполнения этапов разработки и применить соответствующие часовые тарифные ставки специалистов. В разработке системы принимали участие два специалиста: инженер-разработчик (исполнитель) и научный руководитель (консультант).

Общая трудоемкость работ распределяется следующим образом:

1. Инженер-разработчик: 120 часов. Включает сбор и подготовку датасета, обучение нейросетевой модели YOLOv8, программную реализацию модуля связи с 3D-принтером, разработку графического интерфейса пользователя (GUI) и проведение тестирования.

2. Научный руководитель: 20 часов. Включает постановку задачи, корректировку алгоритмов, контроль хода работы и проверку результатов.

Расчет часовых ставок произведен на основе анализа рынка труда в Российской Федерации за 2024 год [10], [11]. Для расчета принята стандартная продолжительность рабочего месяца – 160 часов.

Средняя заработная плата инженера-программиста (уровень стажер/junior) составляет примерно 56 000 руб./мес. Часовая ставка рассчитывается как:

$$C_{\text{исп}} = \frac{56000 \text{ руб}}{160 \text{ ч}} = 350 \text{ руб./ч}$$

Средняя заработная плата научного сотрудника/доцента (к.т.н.) составляет примерно 112 000 руб./мес. Часовая ставка рассчитывается как:

$$C_{\text{рук}} = \frac{112000 \text{ руб}}{160 \text{ ч}} = 700 \text{ руб./ч}$$

Таблица 4.1 – План работ исполнителя

№	Этапы работ	Трудоем- кость (чел/час)	Ставка (руб./час)	Сумма (руб.)
1	Аналитический и подготовительный этап (сбор литературы, настройка оборудования)	10	350	3 500
2	Формирование и разметка датасета (сбор данных, аннотирование, балансировка)	20	350	7 000
3	Обучение модели и разработка модуля связи с принтером (прототипирование)	35	350	12 250
4	Проектирование и разработка графического интерфейса (GUI PyQt6)	20	350	7 000
5	Интеграция компонентов, отладка и финальное тестирование	25	350	8 750
6	Оформление пояснительной записки и подготовка материалов к защите	10	350	3 500
	ИТОГО	120		42 000

Таблица 4.2 – План работ научного руководителя

№	Этапы работ	Трудоем- кость (чел/час)	Ставка (руб./час)	Сумма (руб.)
1	Постановка задачи, формирование технического задания и утверждение плана работы	4	700	2 800
2	Консультации по выбору архитектуры нейросети и методов предварительной обработки данных	6	700	4 200
3	Промежуточный контроль результатов обучения модели и логики управления	6	700	4 200
4	Рецензирование пояснительной записки, нормоконтроль и подготовка к защите	4	700	2 800
ИТОГО		20		14 000

Данные о трудоемкости выполнения работ указаны в человеко-часах и приведены в таблицах 4.1 и 4.2 соответственно.

Трудозатраты исполнителя:

$$T_{\text{исп}} = 120 \text{ чел/час}$$

Трудозатраты научного руководителя:

$$T_{\text{рук}} = 20 \text{ чел/час}$$

Таким образом, общая трудоемкость выполнения выпускной квалификационной работы составила 140 чел/час, из которых 120 часов затрачено непосредственно на проектирование, программную реализацию и тестирова-

ние системы исполнителем, а 20 часов – на научное руководство и консультационную поддержку. Суммарные затраты на оплату труда специалистов, участвующих в разработке, составляют 56 000 рублей.

4.2 Отчисления на социальные нужды

На основе данных о трудоемкости работ и ставках исполнителей, участвующих в проекте, рассчитываются расходы на заработную плату. Процент отчислений на социальные нужды составляет 30,2% (ст. 425 НК РФ, в 2025 г. ставка сохранена на уровне 30% + 0,2% на травматизм для IT-разработок) [12]. Также в расчет принимается процент дополнительной заработной платы, равный 8,3%.

Основная заработная плата рассчитывается по формуле:

$$З_{\text{осн.з./пл}} = T_{\text{исп}} * C_{\text{исп}} + T_{\text{рук}} * C_{\text{рук}},$$

где T_i – время, затраченное i -м участником на проведение работ (в часах);

C_i – ставка i -го участника (в руб./час)

$$З_{\text{осн.з./пл}} = 120 * 350 + 20 * 700 = 42000 + 14000 = 56000 \text{ руб.}$$

Дополнительная заработная плата составляет:

$$З_{\text{доп.з./пл}} = З_{\text{осн.з./пл}} * \frac{H_{\text{доп}}}{100}$$

$$З_{\text{доп.з./пл}} = 56000 * 0,083 = 4648 \text{ руб.}$$

Отчисления на социальные нужды от основной и дополнительной заработной платы равны:

$$З_{\text{соц}} = (З_{\text{осн.з./пл}} + З_{\text{доп.з./пл}}) * \frac{H_{\text{соц}}}{100}$$

$$З_{\text{соц}} = (56000 + 4648) * 0,302 = 18315,70 \text{ руб.}$$

Таким образом, сумма отчислений на социальные нужды, рассчитанная от фонда оплаты труда с учетом дополнительной заработной платы, составила

18 315,70 рублей. Данные отчисления включают взносы в Пенсионный фонд, Фонд социального страхования и Фонд обязательного медицинского страхования в соответствии с действующим законодательством РФ (НК РФ ст. 425–427). Полученная величина учитывается при формировании полной сметной стоимости разработки программного обеспечения.

4.3 Материалы

Основной расходный материал для тестовой печати – PLA-пластик. Дефекты создавались на 5 тестовых моделях, в том числе 2 модели были остановлены автоматически, 3 – доработаны.

Таблица 4.3 – Затраты на материалы

Материал	Кол-во (кг)	Цена за кг (руб.)	Сумма (руб.)
PLA-пластик [13]	0,8	1300	1040
Транспортно-заготовительные расходы (10%)			104
Итого			1144

Прямые материальные затраты на проведение экспериментальной части работы, включая расходы на транспорт и материалы, составили 1144 рубля.

4.4 Амортизационные отчисления

В процессе разработки и тестирования системы использовалось следующее оборудование: персональный компьютер, 3D-принтер Anycubic Kobra 2 Neo и веб-камера Logitech C310.

Стоимость оборудования принята на основе среднерыночных цен в розничной сети DNS на момент проведения работ:

- Персональный компьютер – 50 000 руб.
- 3D-принтер Anycubic Kobra 2 Neo – 13 500 руб. [14]
- Веб-камера Logitech C310 – 2 499 руб. $\approx$ 2 500 руб. [15]

Сумма амортизации за год рассчитывается по формуле:

$$A_{\Gamma} = K_{об} * \frac{H_{AM}}{100},$$

где $K_{об}$ – стоимость оборудования; H_{AM} – норма амортизации.

Согласно постановлению правительства РФ от 01.01.2002 №1 (ред. от 28.04.2018) «О Классификации основных средств, включаемых в амортизационные группы» персональные компьютеры и печатающие устройства к ним, относятся ко второй группе. Срок полезного использования оборудования равен от двух до трех лет включительно. Срок эксплуатации три года. Норма амортизации $H_{AM} = 33,3\%$ [18].

Общая стоимость оборудования:

$$K_{об} = 50000 + 13500 + 2500 = 66000 \text{ руб.}$$

Годовая амортизация:

$$A_{\Gamma} = 66000 * \frac{33,3}{100} = 21\,978 \text{ руб.}$$

Сумма амортизации за рабочий день равна:

$$A_{д} = \frac{A_{\Gamma}}{N}$$

В 2026 году количество рабочих дней составляет 247, соответственно:

$$A_{д} = \frac{21\,978}{247} = 88,98 \text{ руб.}$$

Амортизация оборудования за время проектирования дипломного проекта (исходя из трудоемкости 120 часов, что составляет 15 рабочих дней по 8 часов):

$$A_{вкр} = A_{д} * 15 = 1334,7 \text{ руб.}$$

Таблица 4.4 – Расчет амортизационных отчислений

Оборудование	Стоимость (руб.)	Годовые амортизационные отчисления (руб.)	Время работы (дни)	Амортизация за ВКР (руб.)
ПК	50 000	16 650	15	1 011,30
Anycubic Kobra 2 Neo	13 500	4 495,5	15	272,55
Logitech C310	2 500	832,5	15	50,85
Итого	66 000	21 978	15	1 334,7

Таким образом, сумма амортизационных отчислений за оборудование, использованное при выполнении выпускной квалификационной работы, составила 1334,7 рублей. Данная величина отражает износ основных средств за период разработки и включается в сметную стоимость проекта.

4.5 Услуги сторонних организаций

Для выполнения выпускной квалификационной работы потребовался доступ к сети Интернет для поиска технической документации, загрузки библиотек программного обеспечения, работы с репозиториями и обмена данными с научным руководителем.

Расчет затрат произведен исходя из стоимости тарифного плана домашнего интернета ООО «Ростелеком» и срока выполнения проекта. Период активной фазы разработки составил 3 месяца. Стоимость абонентской платы принята равной 700 руб./мес. [16].

Таблица 4.5 – Затраты по работам, выполняемым сторонними организациями

Услуга	Объем	Цена (руб.)	Сумма (руб.)
Доступ в Интернет (месяц)	3 мес.	700	2100
Итого			2100

4.6 Накладные расходы

Составляют 20% от суммы прямых затрат основной и дополнительной заработной платы основных исполнителей (расходы на электроэнергию, вспомогательные инструменты, канцелярию, техобслуживание) [17].

$$(56\,000 + 4648) \times 0,20 = 12129,6 \text{ руб.}$$

4.7 Смета затрат

Смета затрат представляет собой плановый документ, определяющий совокупность всех расходов, необходимых для реализации проекта по разра-

ботке системы автоматического обнаружения дефектов. В структуру себестоимости входят прямые затраты (материалы, оплата труда, амортизация оборудования, услуги связи) и косвенные расходы (накладные расходы).

Расчет общей стоимости выполнен на основе трудоемкости работ, рыночных цен на оборудование и материалы, а также действующих нормативов отчислений. Итоговая смета представлена в таблице 4.6.

Таблица 4.6 – Смета затрат на ВКР

№	Наименование статьи	Сумма, руб.	Структура себестоимости, (%)
1	Материалы и транспорт	1 144,00	1,20
2	Расходы на оплату труда (основная и дополнительная заработная плата)	60 648,00	63,39
3	Отчисления на социальные нужды	18 315,70	19,14
4	Расходы на амортизацию ПК и оргтехники	1 334,70	1,40
5	Расходы на услуги сторонних организаций (Интернет)	2 100,00	2,19
6	Накладные расходы (20% от фонда оплаты труда)	12 129,60	12,68
	ИТОГО:	95 672,00	100,00

Таким образом, себестоимость выпускной квалификационной работы составляет 95 672 рублей.

Выводы по четвертой главе

В данном разделе была рассчитана себестоимость разработки системы автоматического обнаружения дефектов при 3D-печати с точки зрения вложения финансовых ресурсов.

В процессе анализа экономической составляющей проекта были рассчитаны расходы на основную и дополнительную заработную плату исполнителей, отчисления на социальные нужды, затраты на материалы (расходные пластики), амортизацию используемого оборудования (ПК, 3D-принтер, камера), а также расходы на услуги сторонних организаций (доступ к сети Интернет) и накладные расходы.

Исходя из выполненного расчета, полная сметная стоимость конечного продукта составляет 95 672 рублей. Структура затрат показывает, что проект относится к трудоемким разработкам: максимальную долю в себестоимости занимают расходы на оплату труда (63,39%), отчисления на социальные нужды (19,14%) и накладные расходы (12,68%). Доля прямых материальных затрат и амортизации оборудования незначительна и составляет менее 3% в сумме, что характерно для проектов в сфере программного обеспечения.

Экономическая эффективность внедрения разработанной системы обусловлена снижением процента брака при аддитивном производстве. Практические испытания показали, что автоматическая остановка печати при обнаружении дефекта экструзии позволяет экономить в среднем до 0,3 кг пластика (≈ 390 руб.) и электроэнергию на каждом предотвращенном бракованном изделии. При условии интенсивной эксплуатации оборудования и средней частоте возникновения дефектов 1 раз в 10 часов печати, срок окупаемости затрат на разработку при использовании системы на одном принтере составляет 3–4 месяца. При масштабировании решения на парк из 3–5 единиц оборудования срок окупаемости сокращается до менее чем одного месяца, что подтверждает высокую экономическую целесообразность внедрения разработанного программного комплекса.

ЗАКЛЮЧЕНИЕ

В выпускной квалификационной работе решена задача разработки автономной системы визуального контроля качества FDM-печати на основе компьютерного зрения. Цель работы достигнута, все поставленные задачи выполнены в полном объеме.

В процессе анализа предметной области был выявлен ряд существенных недостатков и ограничений имеющихся решений для мониторинга 3D-печати. Облачные сервисы зависят от интернета, AI-камеры не всегда целесообразны ввиду дорогой себестоимости, а open-source альтернативы сложно интегрировать. При сравнении архитектур сверточных сетей лучшей признали легковесную модель YOLOv8n-cls (2,8 млн параметров). Она оптимально сочетает точность классификации и вычислительную эффективность.

Для обучения модели сформирован специализированный датасет объемом 46 664 изображения, сбалансированный и разбит на два класса. Применен пайплайн предобработки, включающий центральный квадратный кроп, конвертацию в градации серого. В связи с необходимостью расширения датасета была применена аугментация данных: случайное масштабирование, поворот $\pm 15^\circ$, сдвиг $\pm 10\%$, коррекция экспозиции $\pm 5\%$, добавление шума. Гиперпараметры подобрали эмпирически на целевой платформе. В результате дообучения достигнута точность классификации 99,71%, точность обнаружения дефектов, в свою очередь, составила 99,82%. Разрыв между train/loss и val/loss не превысил 0,5%, что подтвердило отсутствие переобучения и высокую обобщающую способность модели.

Разработан программный комплекс на базе Python и PyQt6, реализующий замкнутый контур автоматического управления. Архитектура приложения построена на событийно-ориентированной многопоточной модели: обработка видеопотока и инференс нейросети вынесены в изолированный поток CameraThread, обмен данными с микроконтроллером осуществляется через

асинхронный таймер, а главный поток отвечает исключительно за отрисовку интерфейса и обработку пользовательских событий. Интегрировали модуль последовательного обмена по USB/UART, изучили синтаксис команд G-кода. Для безопасной остановки печати вместо аварийной M112 применили M25 (программная пауза). Такой подход сохраняет позицию экструдера и дает возможность возобновить печать командой M24. Внедрены механизмы защиты от ложных срабатываний: пороговая фильтрация уверенности (по умолчанию $\geq 80\%$), кулдаун повторных команд (2,0 с) и цифровой зум области анализа.

Интеграционное тестирование подтвердило работоспособность системы в условиях, приближенных к реальным. Частота анализа кадров составляет 7,5–8,0 FPS, общая задержка контура (захват → инференс → отправка команды) не превышает 250 мс. На процессоре AMD Ryzen 5 5600U средняя загрузка CPU удерживается в диапазоне 40–55%, потребление оперативной памяти стабилизировано на уровне ~300 МБ. Система демонстрирует отказоустойчивость: корректно обрабатывает обрывы USB-соединения, переполнение буферов и вариации освещения, не прерывая работу графического интерфейса.

Экономическое обоснование проекта показало, что полная себестоимость разработки программного комплекса составляет 95 672 рублей. Структура затрат характерна для программных разработок: основную долю занимают расходы на оплату труда и отчисления. Расчетный срок окупаемости внедрения системы на одном FDM-принтере составляет 3–4 месяца за счет предотвращения расхода филамента и электроэнергии при браке. При масштабировании на парк из 3–5 устройств срок окупаемости сокращается до менее чем одного месяца, что подтверждает высокую экономическую целесообразность проекта.

Практическая значимость работы заключается в создании готового к эксплуатации программного продукта, не требующего облачной инфраструктуры, сложной настройки Linux-среды или специализированного оборудования. Система обладает интуитивным интерфейсом, поддерживает гибкую

настройку порогов срабатывания и обеспечивает полную трассируемость событий через двойное логирование.

Несмотря на то, что все поставленные задачи были выполнены, предусмотрено дальнейшее наращивание функционала. Одним из направлений дальнейшего развития является добавление в систему возможности подключения нескольких камер, для более качественного и оперативного выявления дефектов, планируется расширить количество поддерживаемых прошивок, например, Klipper через WebSocket. Также перспективной научить модель обучаться на новых данных прямо во время работы, без остановки производства. В добавок можно сделать мобильное приложение – чтобы оператор получал сигналы удаленно. В таком виде программа уже готова к внедрению на небольших производствах с послойным наплавлением. Это сделает печать надежнее и уменьшит издержки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Jocher G., Chaurasia A., Qiu J. Ultralytics YOLOv8 [Электронный ресурс]. – URL: <https://github.com/ultralytics/ultralytics> (дата обращения: 29.04.2026).
2. PySerial. PySerial Documentation [Электронный ресурс]. – URL: <https://pyserial.readthedocs.io/> (дата обращения: 29.04.2026).
3. Marlin Firmware. Documentation [Электронный ресурс]. – URL: <https://marlinfw.org/docs/> (дата обращения: 29.04.2026).
4. Nimbus200. 3D printing errors [Электронный ресурс] / Nimbus200. – URL: <https://www.kaggle.com/datasets/nimbus200/3d-printing-errors> (дата обращения: 25.04.2026).
5. justin900429. 3D-Printer Defected Dataset [Электронный ресурс]. – URL: <https://www.kaggle.com/datasets/justin900429/3d-printer-defected-dataset> (дата обращения: 20.05.2026).
6. Roboflow. Computer Vision Dataset Tools [Электронный ресурс]. – URL: <https://roboflow.com/> (дата обращения: 25.04.2026).
7. Ultralytics. YOLOv8 Documentation [Электронный ресурс]. – URL: <https://docs.ultralytics.com/> (дата обращения: 29.04.2026).
8. OpenCV. OpenCV Documentation [Электронный ресурс]. – URL: <https://docs.opencv.org/> (дата обращения: 29.04.2026).
9. PyQt6. Official Documentation [Электронный ресурс]. – URL: <https://www.riverbankcomputing.com/static/Docs/PyQt6/> (дата обращения: 29.04.2026).
10. Росстат. Среднемесячная номинальная начисленная заработная плата работников по полному кругу организаций в целом по экономике РФ в 2024 г. [Электронный ресурс]. – URL: https://rosstat.gov.ru/labor_market_employment_salaries (дата обращения: 29.04.2026).

11. ГородРабот.ру. Зарплата Инженера-программиста в России [Электронный ресурс]. - URL: <https://russia.gorodrabot.ru/salaries/inzhener-programmist?y=2026> (дата обращения: 29.04.2026).

12. Налоговый кодекс Российской Федерации (часть вторая) от 05.08.2000 № 117-ФЗ (ред. от 29.12.2024) - ст. 425.

13. Ozon. Катушка PLA-пластика eSUN 1,75 мм, 1 кг, черный (black) [Электронный ресурс]. - URL: <https://www.ozon.ru/product/katushka-pla-plastika-esun-1-75-mm-1kg-chernaya-black-1541037404> (дата обращения: 25.04.2026).

14. DNS. 3D-принтер Anycubic Kobra 2 Neo [Электронный ресурс]. - URL: <https://www.dns-shop.ru/product/e65f49d52c61d9cb> (дата обращения: 29.04.2026).

15. DNS. Веб-камера Logitech C310 [Электронный ресурс]. - URL: <https://www.dns-shop.ru/product/8ca6732d2511ed20> (дата обращения: 29.04.2026).

16. Ростелеком. Тариф «Технологии доступа» : 200 Мбит/с - 700 руб./мес. [Электронный ресурс]. - URL: <https://spb.rt.ru/> (дата обращения: 29.04.2026).

17. Минэкономразвития РФ. Рекомендации по расчету накладных расходов в НИОКР : письмо от 21.12.2023 № Д23и-4567.

18. Постановление Правительства РФ от 01.01.2002 № 1 (ред. от 28.04.2018) «О классификации основных средств, включаемых в амортизационные группы».